Fusion RAG — Production-Grade Reverse Engineering & Architectural Blueprint

- 1 Fusion RAG: Production-Grade Reverse Engineering & Architectural Blueprint
  - 1.0.1 Single Source of Truth, System Design Deep Dive & Master Technical Interview Guide
  - 1.1 Document Overview & How to Use This Guide
- 2 Table of Contents
- 3 Part I: Executive Overview & High-Level System Architecture
  - 3.1 1. System Vision, Problem Statement & Core Value Proposition
    - 3.1.1 1.1 The Industry Problem
    - 3.1.2 1.2 The Fusion RAG Solution
    - 3.1.3 1.3 Core Concept "Cheat Sheet" for Freshers & Interviewees
  - 3.2 2. End-to-End System Topology & High-Level Architecture Diagram
  - 3.3 3. Architectural Selection Matrix & Tradeoff Analysis
  - 3.4 4. Complete Project Directory Layout & Module Responsibilities
- 4 Part II: End-to-End Sequence Diagrams & Step-by-Step Data Flows
  - 4.1 5. Document Ingestion, Parsing, Chunking & Vector Indexing Sequence
  - 4.2 6. Real-Time Chat & SSE Streaming Lifecycle Sequence
  - 4.3 7. Production Hybrid Retrieval Pipeline Sequence (Dense + Sparse + RRF + MMR + Reranker)
  - 4.4 8. Hierarchical Multi-Agent DAG Planning & Execution Wave Sequence
  - 4.5 9. Authentication, Token Rotation, CSRF & Replay Attack Defense Sequence
  - 4.6 10. Transactional Outbox Pattern & Event Bus Sequence
  - 4.7 11. Knowledge Graph Extraction & Multi-Hop Traversal Sequence
- 5 Part III: Specialist Agent, Tool Calling & Dual-Layer Orchestration Architecture
  - 5.1 12. Dual Orchestration Layers: Adaptive RAG vs. Hierarchical Multi-Agent DAG
  - 5.2 13. MasterAgent: Hierarchical Planner, Replanner & Evidence Gating
    - 5.2.1 13.1 Dynamic DAG Planning & Prompt Engineering
    - 5.2.2 13.2 Deterministic Heuristic Fallback Planner (`_plan_heuristic`)
    - 5.2.3 13.3 Multi-Agent Internal Pydantic Contracts
    - 5.2.4 13.4 Evidence Relevance Gate Algorithm
  - 5.3 14. AgentExecutionEngine: Topological Wave Executor & Concurrency Controller
  - 5.4 15. Dedicated Adaptive RAG Orchestration Framework (`app/rag/orchestration/`)
  - 5.5 16. ResearchAgent & ResearchPlanner: Sub-Query Decomposition & KB Retrieval
  - 5.6 17. DocumentAgent: Explicit Document & Chunk Inspection Specialist
  - 5.7 18. AnalysisAgent: Deterministic Math, AST Sandboxing, SymPy & MathEngine
  - 5.8 19. WebSearchAgent: Real-Time Multi-Provider Web Search Cascade
  - 5.9 20. TableAgent: Tabular Data Extraction & Column Aggregations
  - 5.10 21. VisionAgent: Multimodal Visual Reasoning & Chart Inspection
  - 5.11 22. GraphAgent: Knowledge Graph Traversal & Entity Relationship Reasoning
  - 5.12 23. SynthesisAgent: Context Grounding, Citation Tagging & Refusal Faithfulness
  - 5.13 24. ResultEvaluator: Task Quality Scoring & Adaptive Replanning Engine
  - 5.14 25. PlanValidator: Schema, Cycle Detection & Constraint Enforcement
  - 5.15 26. TaskRouter & CapabilityRegistry: Capability-Aware Cost/Latency Routing
  - 5.16 27. Tool Calling, Function Declaration Schemas & Discovery Engine

        - 5.17 28. Multi-Agent & Chat Operational Subsystems: Keyset Pagination, Memory & LLM Failover
- 6 Part IV: Relational Database Schema, Data Models & Repository Layer (All 25 Tables & 14 Repositories)
    - 6.1 29. Relational Architecture, Entity-Relationship Diagram (ERD) & Bounded Contexts
        - 6.1.1 29.1 Composite Foreign Keys, Partial Indexes & SQLite Variants
    - 6.2 30. Domain Repository Architecture & Abstract Interfaces
    - 6.3 31. Authentication & Security Context (5 Tables & Repositories)
        - 6.3.1 31.1 `users` Table
        - 6.3.2 31.2 `refresh_sessions` Table
        - 6.3.3 31.3 `user_encryption_keys` Table
        - 6.3.4 31.4 `email_verification_tokens` Table
        - 6.3.5 31.5 `user_settings` Table
    - 6.4 32. Chat & Conversation Context (3 Tables & Repositories)
        - 6.4.1 32.1 `conversations` Table
        - 6.4.2 32.2 `conversation_messages` Table
        - 6.4.3 32.3 `conversation_summaries` Table
    - 6.5 33. Document Ingestion, Chunks & Storage Context (9 Tables & Repositories)
        - 6.5.1 33.1 `documents` Table
        - 6.5.2 33.2 `document_files` Table
        - 6.5.3 33.3 `document_chunks` Table
        - 6.5.4 33.4 `document_embeddings` Table
        - 6.5.5 33.5 `document_elements` Table
        - 6.5.6 33.6 `processing_config_versions` Table
        - 6.5.7 33.7 `document_processing_runs` Table
        - 6.5.8 33.8 `document_processing_jobs` Table
        - 6.5.9 33.9 `durable_cleanup_jobs` Table
    - 6.6 34. Knowledge Graph Context (4 Tables & Repositories)
        - 6.6.1 34.1 `graph_entities` Table
        - 6.6.2 34.2 `graph_relationships` Table
        - 6.6.3 34.3 `graph_relationship_evidence` Table
        - 6.6.4 34.4 `graph_entity_mentions` Table
    - 6.7 35. Transactional Outbox Context (1 Table) & Automated Evaluation Context (3 Tables)
        - 6.7.1 35.1 `outbox_events` Table
        - 6.7.2 35.2 `evaluation_runs` Table
        - 6.7.3 35.3 `evaluation_results` Table
        - 6.7.4 35.4 `benchmark_runs` Table
- 7 Part V: Security, Reliability, Caching & Distributed Infrastructure
    - 7.1 36. Envelope Encryption, Key Management & Crypto-Shredding (AES-256-GCM + DEK/KEK)
    - 7.2 37. Malware & Antivirus Scanning (ClamAV TCP Streaming + EICAR)
    - 7.3 38. Prompt Injection Defense, Intent Guardrails & AST Sandboxing
    - 7.4 39. Multi-Tier Caching (L1 Memory + L2 Redis) & Distributed Heartbeat Locking
    - 7.5 40. Reliability Framework: Bulkheads, Circuit Breakers, Exponential Jitter Retries & Timeouts
    - 7.6 41. Transactional Outbox Pattern, Event Bus & JobDispatcher Engine
    - 7.7 42. Storage Provider Abstraction & Two-Phase Durable Cleanup Worker
    - 7.8 43. Transactional Email, Request Idempotency, Pricing Engine & Resource Quotas
    - 7.9 44. Application Lifespan, Startup Hooks & Middleware Cardinality Defense

- 8 Part VI: Document Ingestion, Advanced Chunking & Retrieval Intelligence
  - 8.1 45. Ingestion Worker Lease Protocol, Multi-Phase Pipeline & Durable Reaper
    - 8.1.1 45.1 Multi-Phase Ingestion State Machine (`ProcessDocumentJobHandler`)
    - 8.1.2 45.2 Ingestion Lease, Heartbeat & Durable Reaper
  - 8.2 46. Advanced Chunking Strategies: Recursive, Parent-Child & Sentence-Window
  - 8.3 47. Empty Knowledge Base Index Cache & Weighted RRF Fusion
  - 8.4 48. Document Parsers, Multimodal OCR, Tables & Metadata Extractors
  - 8.5 49. Query Intelligence: Classification, Decomposition, Expansion, HyDE & ContextSelector
  - 8.6 50. Ingestion-Time NER, Relationship Extraction & BFS Knowledge Graph Retrieval
  - 8.7 51. Adaptive RAG Orchestration State Machine (DIRECT, HYDE, MULTI_HOP, SPECIALIST)
- 9 Part VII: Frontend Architecture, Pyodide WASM & UI Subsystems
  - 9.1 52. Pyodide WebAssembly (WASM) Sandbox Worker Protocol & Managers
  - 9.2 53. Complete Custom Hook Architecture & State Context Providers
    - 9.2.1 53.1 Frontend Context Providers
    - 9.2.2 53.2 Complete Custom Hook Inventory
  - 9.3 54. 7-Stage Progressive Upload Pipeline & State Machine
  - 9.4 55. Application Pages, Route Guards, Projects & Settings Subsystems
  - 9.5 56. UI Component Tree, Axios Interceptors, Stream Recovery & CodeInterpreter
- 10 Part VIII: Complete REST & SSE API Endpoint Catalog (Exact Codebase Routes & DTOs)
  - 10.1 57. Exhaustive Route Specifications Across All 10 Routers & Root App
    - 10.1.1 57.1 Core Pydantic DTO Schema Definitions
- 11 Part IX: Production Observability, Quality Assurance & Evaluation Suite
  - 11.1 58. OpenTelemetry Tracing, Prometheus Metric Registry & Telemetry Sanitizer
    - 11.1.1 58.1 Complete Prometheus Metric Registry
  - 11.2 59. Multi-Agent Observability & Quality Metrics (AgentObservabilityService)
  - 11.3 60. Comprehensive Evaluation Framework: 6 Evaluators, 5 Runners & 33 Metrics
    - 11.3.1 60.1 Dedicated Evaluator Services (`app/evaluation/services/`)
    - 11.3.2 60.2 Evaluation Runners (`app/evaluation/runners/`)
    - 11.3.3 60.3 Complete Catalog of All 33 Registered Metrics in `backend/app/evaluation/metrics/registry.py`
  - 11.4 61. Golden Corpora, Benchmark Profiles, Drift Tracking & Report Generators
- 12 Part X: DevOps, CI/CD Automation, Operational Tooling & Edge Deployment
  - 12.1 62. The 4 GitHub Actions Workflows (CI, E2E Playwright, Eval Gate, Nightly Benchmark)
  - 12.2 63. Operational CLI Scripts, Backup/Restore, Test Taxonomy & Verification
    - 12.2.1 63.1 Complete 12 Operational CLI Scripts (`backend/scripts/`)
    - 12.2.2 63.2 Complete Test Suite Taxonomy
  - 12.3 64. Docker Compose Topologies, Alembic Lineage, Nginx Reverse Proxy & Runbooks
    - 12.3.1 64.1 Alembic Database Migration Lineage
    - 12.3.2 64.2 Observability Ports & Edge Configuration
- 13 Part XI: Real-World Engineering Case Studies (DaveAI Production Challenges)
  - 13.1 65. Case Study 1: The Asynchronous Streaming Transaction Race Condition (EarlyCommit Fix)
  - 13.2 66. Case Study 2: Database Connection Pool Starvation during Streaming Responses
  - 13.3 67. Case Study 3: Cascading Timeouts & Topological Deadlocks in Multi-Agent Execution
  - 13.4 68. Case Study 4: Multi-Key Rate-Limit Cascades & High-Traffic Failover

- 14 Part XII: Master Technical Interview Preparation Guide & Environment Catalog
  - 14.1 69. Technical Narrative Framework (STAR Format for Fusion RAG)
  - 14.2 70. Technical Architecture & Distributed Systems Interview Q&A
    - 14.2.1 Q1: Walk us through how you design asynchronous APIs using FastAPI and Python to handle high-concurrency workloads.
    - 14.2.2 Q2: How have you utilized Redis and WebSockets/SSE in your past projects to build real-time systems? What challenges did you face with state management or message delivery?
    - 14.2.3 Q3: What is your approach to structuring and scaling relational databases like PostgreSQL for production-oriented applications?
  - 14.3 71. Rapid Revision Flashcard Matrix (Numbers, Formulas, DB Indexes & Shell Commands)
    - 14.3.1 71.1 Key System Constants & Production Defaults
    - 14.3.2 71.2 Essential PostgreSQL Indexes
  - 14.4 72. Exhaustive Environment Configuration Variable Catalog (All 150+ Parameters)
- 15 Part XIII: Production Multi-Agent & Multimodal Verification Master Index
  - 15.1 73. 7-Specialist Runtime Dependency Injection Matrix
  - 15.2 74. Asynchronous Document Enrichment Pipeline (`DocumentEnrichmentService`)
  - 15.3 75. Adversarial Dynamic DAG Execution Plans (Scenarios 1–6)
    - 15.3.1 Scenario 1: Factual Document Text Query
    - 15.3.2 Scenario 2: Tabular Arithmetic / CAGR Calculation
    - 15.3.3 Scenario 3: Visual Chart Trend Inspection
    - 15.3.4 Scenario 4: Knowledge Graph Relationship Query
    - 15.3.5 Scenario 5: Hybrid Document + Live Web + Difference
    - 15.3.6 Scenario 6: LLM-Generated Dynamic Complex Multi-Specialist DAG
  - 15.4 76. Real End-to-End Chat Retrieval & Multimodal Verification Trace
  - 15.5 77. System-Wide Quality & Security Verification Matrix

1 Fusion RAG: Production-Grade Reverse Engineering & Architectural Blueprint

1.0.1 Single Source of Truth, System Design Deep Dive & Master Technical Interview Guide

1.1 Document Overview & How to Use This Guide

This document is the **definitive, exhaustive technical specification, data flow blueprint, database manual, frontend breakdown, DevOps manual, and architectural guide** for the **Fusion RAG Platform**. It is engineered to enable a Principal Software Engineer, Staff Architect, or Technical Lead to understand, maintain, scale, or recreate the entire distributed system from scratch without external documentation or AI assistance.

```
================================================================================================================
                                        SYSTEM QUICK REFERENCE
================================================================================================================
* Core Framework:       FastAPI (Python 3.12+ ASGI) + Uvicorn + Pydantic v2
* Primary Database:     PostgreSQL 16 (asyncpg + SQLAlchemy 2.0 async ORM, 25 Relational Tables, 14 Domain Persistence Components)
* Vector Search Engine: Qdrant v1.19.0 (HNSW ANN, 768-dim Cosine, Payload Tenant Fencing)
* Sparse Text Search:   PostgreSQL Full-Text Search (tsvector + ts_rank_cd + GIN Indexes)
* In-Memory Cache/Queue:Redis 7 (redis.asyncio, Redis Streams Task Queue, Redis PubSub Event Bus, L1/L2 Cache)
* Dense Embeddings:     Jina AI API (jina-embeddings-v3, Matryoshka Representation, 768-dim)
* Cross-Encoder Rerank: Jina AI API (jina-reranker-v2-base-multilingual, Async Semaphores)
* Primary LLM Engine:   Google Gemini API (gemini-3.5-flash-lite via Multi-Key Workload Isolation & Failover Proxy)
* Dual Orchestration:   1. Lightweight Adaptive RAG (app/rag/orchestration/) for single-turn intent routing
                        2. Full Multi-Agent DAG (app/chat/services/multi_agent/) for complex wave reasoning
* Observability:        OpenTelemetry SDK + Jaeger (OTLP gRPC) + Prometheus + Grafana + PII Sanitizer
* Evaluation Engine:    Ragas + DeepEval + 6 Evaluator Services + 5 Runners + 33 Registered Metrics Suite
* Frontend Architecture:React 18 + Vite 5 + TypeScript + Vanilla CSS Tokens + Pyodide WebAssembly (WASM)
* Security & Crypto:    Envelope Encryption (AES-256-GCM + DEK/KEK), ClamAV Antivirus, AST Sandboxing, Intent Guardrails
* Reliability:          Bulkheads, Circuit Breakers, Exponential Full-Jitter Retries, Outbox Pattern
* DevOps & CI/CD:       4 GitHub Actions Workflows (CI, E2E Playwright, Eval, Nightly Benchmark) + 12 CLI Scripts + Nginx SSE
================================================================================================================
```

2 Table of Contents

- Part I: Executive Overview & High-Level System Architecture
  - 1. System Vision, Problem Statement & Core Value Proposition
  - 2. End-to-End System Topology & High-Level Architecture Diagram
  - 3. Architectural Selection Matrix & Tradeoff Analysis
  - 4. Complete Project Directory Layout & Module Responsibilities
- Part II: End-to-End Sequence Diagrams & Step-by-Step Data Flows
  - 5. Document Ingestion, Parsing, Chunking & Vector Indexing Sequence
  - 6. Real-Time Chat & SSE Streaming Lifecycle Sequence
  - 7. Production Hybrid Retrieval Pipeline Sequence (Dense + Sparse + RRF + MMR + Reranker)
  - 8. Hierarchical Multi-Agent DAG Planning & Execution Wave Sequence
  - 9. Authentication, Token Rotation, CSRF & Replay Attack Defense Sequence
  - 10. Transactional Outbox Pattern & Event Bus Sequence
  - 11. Knowledge Graph Extraction & Multi-Hop Traversal Sequence
- Part III: Specialist Agent, Tool Calling & Dual-Layer Orchestration Architecture
  - 12. Dual Orchestration Layers: Adaptive RAG vs. Hierarchical Multi-Agent DAG
  - 13. MasterAgent: Hierarchical Planner, Replanner & Evidence Gating
  - 14. AgentExecutionEngine: Topological Wave Executor & Concurrency Controller
  - 15. Dedicated Adaptive RAG Orchestration Framework (`app/rag/orchestration/`)
  - 16. ResearchAgent & ResearchPlanner: Sub-Query Decomposition & KB Retrieval
  - 17. DocumentAgent: Explicit Document & Chunk Inspection Specialist
  - 18. AnalysisAgent: Deterministic Math, AST Sandboxing, SymPy & MathEngine
  - 19. WebSearchAgent: Real-Time Multi-Provider Web Search Cascade
  - 20. TableAgent: Tabular Data Extraction & Column Aggregations
  - 21. VisionAgent: Multimodal Visual Reasoning & Chart Inspection
  - 22. GraphAgent: Knowledge Graph Traversal & Entity Relationship Reasoning
  - 23. SynthesisAgent: Context Grounding, Citation Tagging & Refusal Faithfulness
  - 24. ResultEvaluator: Task Quality Scoring & Adaptive Replanning Engine
  - 25. PlanValidator: Schema, Cycle Detection & Constraint Enforcement
  - 26. TaskRouter & CapabilityRegistry: Capability-Aware Cost/Latency Routing
  - 27. Tool Calling, Function Declaration Schemas & Discovery Engine
  - 28. Multi-Agent & Chat Operational Subsystems: Access Control, Memory & LLM Failover
- Part IV: Relational Database Schema, Data Models & Repository Layer (All 25 Tables & 14 Repositories)
  - 29. Relational Architecture, Entity-Relationship Diagram (ERD) & Bounded Contexts
  - 30. Domain Repository Architecture & Abstract Interfaces
  - 31. Authentication & Security Context (5 Tables & Repositories)
  - 32. Chat & Conversation Context (3 Tables & Repositories)
  - 33. Document Ingestion, Chunks & Storage Context (9 Tables & Repositories)
  - 34. Knowledge Graph Context (4 Tables & Repositories)
  - 35. Transactional Outbox Context (1 Table) & Automated Evaluation Context (3 Tables)
- Part V: Security, Reliability, Caching & Distributed Infrastructure
  - 36. Envelope Encryption, Key Management & Crypto-Shredding (AES-256-GCM + DEK/KEK)
  - 37. Malware & Antivirus Scanning (ClamAV TCP Streaming + EICAR)

    - 38. Prompt Injection Defense, Intent Guardrails & AST Sandboxing
    - 39. Multi-Tier Caching (L1 Memory + L2 Redis) & Distributed Heartbeat Locking
    - 40. Reliability Framework: Bulkheads, Circuit Breakers, Exponential Jitter Retries & Timeouts
    - 41. Transactional Outbox Pattern, Event Bus & JobDispatcher Engine
    - 42. Storage Provider Abstraction & Two-Phase Durable Cleanup Worker
    - 43. Transactional Email, Request Idempotency, Pricing Engine & Resource Quotas
    - 44. Application Lifespan, Startup Hooks & Middleware Cardinality Defense
- Part VI: Document Ingestion, Advanced Chunking & Retrieval Intelligence
    - 45. Ingestion Worker Lease Protocol, Multi-Phase Pipeline & Durable Reaper
    - 46. Advanced Chunking Strategies: Recursive, Parent-Child & Sentence-Window
    - 47. Empty Knowledge Base Index Cache & Weighted RRF Fusion
    - 48. Document Parsers, Multimodal OCR, Tables & Metadata Extractors
    - 49. Query Intelligence: Classification, Decomposition, Expansion, HyDE & ContextSelector
    - 50. Ingestion-Time NER, Relationship Extraction & BFS Knowledge Graph Retrieval
    - 51. Adaptive RAG Orchestration State Machine (DIRECT, HYDE, MULTI_HOP, SPECIALIST)
- Part VII: Frontend Architecture, Pyodide WASM & UI Subsystems
    - 52. Pyodide WebAssembly (WASM) Sandbox Worker Protocol & Managers
    - 53. Complete Custom Hook Architecture & State Context Providers
    - 54. 7-Stage Progressive Upload Pipeline & State Machine
    - 55. Application Pages, Route Guards, Projects & Settings Subsystems
    - 56. UI Component Tree, Axios Interceptors, Stream Recovery & CodeInterpreter
- Part VIII: Complete REST & SSE API Endpoint Catalog (Exact Codebase Routes & DTOs)
    - 57. Exhaustive Route Specifications Across All 10 Routers & Root App
- Part IX: Production Observability, Quality Assurance & Evaluation Suite
    - 58. OpenTelemetry Tracing, Prometheus Metric Registry & Telemetry Sanitizer
    - 59. Multi-Agent Observability & Quality Metrics (AgentObservabilityService)
    - 60. Comprehensive Evaluation Framework: 6 Evaluators, 5 Runners & 33 Metrics
    - 61. Golden Corpora, Benchmark Profiles, Drift Tracking & Report Generators
- Part X: DevOps, CI/CD Automation, Operational Tooling & Edge Deployment
    - 62. The 4 GitHub Actions Workflows (CI, E2E Playwright, Eval Gate, Nightly Benchmark)
    - 63. Operational CLI Scripts, Backup/Restore, Test Taxonomy & Verification
    - 64. Docker Compose Topologies, Alembic Lineage, Nginx Reverse Proxy & Runbooks
- Part XI: Real-World Engineering Case Studies (DaveAI Production Challenges)
    - 65. Case Study 1: The Asynchronous Streaming Transaction Race Condition (EarlyCommit Fix)
    - 66. Case Study 2: Database Connection Pool Starvation during Streaming Responses
    - 67. Case Study 3: Cascading Timeouts & Topological Deadlocks in Multi-Agent Execution
    - 68. Case Study 4: Multi-Key Rate-Limit Cascades & High-Traffic Failover
- Part XII: Master Technical Interview Preparation Guide & Environment Catalog
    - 69. Technical Narrative Framework (STAR Format for Fusion RAG)
    - 70. Technical Architecture & Distributed Systems Interview Q&A
    - 71. Rapid Revision Flashcard Matrix (Numbers, Formulas, DB Indexes & Shell Commands)
    - 72. Exhaustive Environment Configuration Variable Catalog (All 150+ Parameters)
- Part XIII: Production Multi-Agent & Multimodal Verification Master Index
    - 73. 7-Specialist Runtime Dependency Injection Matrix
    - 74. Asynchronous Document Enrichment Pipeline (DocumentEnrichmentService)

◦ 75. Adversarial Dynamic DAG Execution Plans (Scenarios 1–6)
◦ 76. Real End-to-End Chat Retrieval & Multimodal Verification Trace
◦ 77. System-Wide Quality & Security Verification Matrix

---

3 Part I: Executive Overview & High-Level System Architecture

3.1 1. System Vision, Problem Statement & Core Value Proposition

3.1.1 1.1 The Industry Problem

Standard Retrieval-Augmented Generation (RAG) implementations suffer from five primary production pathologies:

1. **Semantic Blindness / Keyword Disconnect**: Pure dense vector retrieval (bi-encoders) fails on exact keyword searches, product codes, SKUs, acronyms, and technical identifiers (e.g., searching for error code `0x80070005` or part number `CR-4096`).
2. **Dense False Positives / High Semantic Overlap**: Dense embeddings retrieve passages with similar grammatical structure or high-level topic alignment that lack the specific factual answer.
3. **Chunk Boundary Truncation**: Rigid chunking breaks tables, sentences, and contextual definitions across chunk boundaries, leading to hallucinated synthesis.
4. **Context Drift in Multi-Turn Dialogues**: Follow-up questions (e.g., *"What about its warranty?"* or *"Can you compare it to the first option?"*) fail because standard RAG embeds the pronoun-laden query directly, searching for "it" instead of the resolved entity.
5. **Slow Time-To-First-Token (TTFT) & Fragility**: Ingestion pipelines block synchronous API threads; single LLM API rate limits cause total application blackouts; unbuffered database writes during streaming cause connection exhaustion.

3.1.2 1.2 The Fusion RAG Solution

Fusion RAG resolves these challenges through an enterprise-grade, asynchronous, multi-tiered architecture:

- **Hybrid Search with Reciprocal Rank Fusion (RRF)**: Merges dense vector HNSW search with PostgreSQL GIN full-text search (FTS) using rank-based reciprocal scoring ($k = 60$).
- **Maximal Marginal Relevance (MMR) & Cross-Encoder Reranking**: Balances information diversity ($\lambda = 0.5$) before passing candidates to a heavy multilingual cross-encoder (`jina-reranker-v2`) bounded by async semaphores.
- **Dual-Layer Orchestration Architecture**:
    1. *Lightweight Adaptive RAG Layer* (`app/rag/orchestration/`): Sub-second routing (`DIRECT`, `HYDE`, `MULTI_HOP`, `SPECIALIST`) with evidence sufficiency gating ($S_{\text{evidence}} \geq 0.70$).
    2. *Full Hierarchical Multi-Agent DAG* (`app/chat/services/multi_agent/`): Decomposes complex multi-domain analytical tasks into topological waves across 7 specialist agents (`RESEARCH`, `DOCUMENT`, `ANALYSIS`, `WEB_SEARCH`, `TABLE`, `VISION`, `GRAPH`) + `SynthesisAgent` grounding layer.
- **Two-Tier Document Model with Scope Isolation**: Strict separation between user-wide persistent Library documents and session-bound Conversation documents with cryptographic multi-tenancy.
- **Zero-PyTorch Cloud-Native Architecture**: Achieves sub-second cold starts and container images under 450 MB by delegating heavy matrix operations to specialized, resilient REST APIs.

3.1.3 1.3 Core Concept "Cheat Sheet" for Freshers & Interviewees

Term	What It Sounds Like	What It Actually Means in Plain English
Bi-Encoder (Dense Search)	Complex neural net	Encodes questions and documents*separately* into 768-dim coordinate vectors so we can do ultra-fast approximate geometric distance search ($O(\log N)$ in Qdrant).
Cross-Encoder (Reranker)	Another neural net	Feeds the question and candidate chunk*together* into a deep multi-head attention layer. Slower ($O(N)$), but understands nuanced word-by-word context with supreme accuracy.
RRF (k = 60)	Complex math formula	A fair leaderboard system: merges dense vector results and keyword results by their rank position (1/(60 + rank)) instead of incompatible raw similarity scores.
MMR (λ = 0.5)	Complex algorithm	"Don't retrieve 5 chunks that repeat the exact same sentence." Balances pure similarity against information diversity so LLM context isn't wasted on duplicate facts.
Matryoshka Embeddings (MRL)	Russian nesting dolls	Embeddings engineered so you can slice the first 256 dimensions out of 768 and retain 95%+ search accuracy while cutting Qdrant RAM and disk storage by 66%.
EarlyCommit Pattern	DB transaction pattern	Committing a newly generated chat session to PostgreSQL*before* starting the token stream, so concurrent frontend attachment uploads don't encounter a 404 race condition.
Topological Wave DAG	Graph theory	Grouping independent tasks together into parallel "waves" so tasks without dependencies run simultaneously, while dependent tasks wait for upstream outputs.

3.2 2. End-to-End System Topology & High-Level Architecture Diagram



Figure — End-to-End System Topology & High-Level Architecture

3.3 3. Architectural Selection Matrix & Tradeoff Analysis

Component	Selected Technology	Alternative Considered	Why Selected (Deep Technical Rationale)	Tradeoffs & Mitigations
API Framework	**FastAPI + Uvicorn (ASGI)**	Django, Flask, Express.js	Native Python`async/await` support for non-blocking I/O; automatic OpenAPI validation via Pydantic v2; ASGI standard supports long-lived HTTP SSE streaming without thread blocking.	Python GIL exists for CPU-bound tasks.*Mitigation*: Offload CPU-heavy text extraction to async workers and MathEngine AST parser.
Relational Database	**PostgreSQL 16 (asyncpg)**	MySQL, MongoDB	Robust ACID compliance; native JSONB for flexible chat metadata; built-in Full-Text Search (`tsvector`, `websearch_to_tsquery`) with GIN indexes eliminates separate Elasticsearch cluster.	Connection overhead per backend process.*Mitigation*: Tuned `async_sessionmaker` connection pool (N = 20, overflow 10, statement cache disabled for PgBouncer).
Vector Database	**Qdrant v1.19.0**	Pinecone, Weaviate, Milvus, pgvector	Written in Rust with ultra-low memory overhead; native payload filtering directly inside HNSW graph index traversal (crucial for multi-tenant`user_id` filtering); fully self-hostable.	Self-managed disk persistence.*Mitigation*: Mounted persistent volume `/qdrant/storage` with automated WAL snapshots.
Message Queue / Cache	**Redis 7 (Streams + Memory)**	Celery + RabbitMQ, Kafka	Single in-memory infrastructure handles sliding-window rate limiting, multi-tier semantic caching, distributed locking, and durable task queues with consumer groups (`XREADGROUP`).	In-memory RAM constraints.*Mitigation*: Configured LRU eviction policy for cache namespaces; persistent AOF for task queue streams.
Dense Embeddings	**Jina Embeddings v3**	OpenAI`text-embedding-3-small`, HuggingFace local	768-dim Matryoshka Representation Learning (MRL) allows truncation to 256/128 dims; asymmetric task adapters (`retrieval.passage` vs `retrieval.query`); zero GPU infrastructure needed.	Third-party REST latency & rate limits.*Mitigation*: 24h Redis embedding cache by SHA-256 hash; async batching (N = 32).
Cross-Encoder Reranker	**Jina Reranker v2 (Multilingual)**	Cohere Rerank, FlashRank local	Joint query-document cross-attention significantly out-ranks bi-encoders; multilingual; REST API eliminates massive local PyTorch RAM footprint (saves >2GB RAM).	Network latency (~150-250ms).*Mitigation*: Bound reranking candidates to top K = 20; concurrency controlled by `asyncio.Semaphore(4)`.
LLM Generation	**Google Gemini (gemini-3.5-flash-lite)**	OpenAI GPT-4o-mini, Claude 3.5 Haiku	Sub-second TTFT (~220ms); massive context window (1M tokens); competitive cost per 1M tokens; native multimodal support for document OCR.	Occasional rate limiting on single API keys.*Mitigation*: Multi-key isolation pattern with `ProviderFailoverProxy`.

3.4 4. Complete Project Directory Layout & Module Responsibilities

```
FusionRAG/                          # Repository root
  prd.md                            # Product Requirements Document
  ARCHITECTURE.md                   # Architecture diagrams and specs
  README.md                         # Quick-start guide
  reverse_engineering.md            # This document (Single Source of Truth)
  docker-compose.yml                # Core services (postgres, redis, qdrant, backend, frontend, nginx)
  docker-compose.observability.yml  # Observability services (jaeger, prometheus, grafana)
  .github/workflows/                # 4 CI/CD workflows (ci, e2e, evaluation, nightly-benchmark)
  deployment/nginx/                 # Nginx configuration (reverse proxy, SSE buffer config)
  observability/                    # Grafana dashboards, Prometheus scrape configs
  docs/                             # Runbooks, Free-tier deployment guides, security checklist
    api/openapi.json                # Static OpenAPI 3.1 specification schema (110 KB)
    architecture/                   # System topology and deployment blueprints
    deployment/                     # Disaster recovery, rollback runbooks, free-tier guides
    security/                       # Vulnerability reporting protocol and crypto manuals

  backend/
    Dockerfile                      # Multi-stage backend build (uv builder + slim runtime)
    pyproject.toml                  # Python dependencies and build metadata
    requirements.txt                # Pinned production dependencies
    alembic.ini                     # Database migration configuration
    alembic/versions/               # 15 versioned migration scripts (including keyset pagination composite indexes)
    scripts/                        # 12 Operational scripts (backup, restore, eval, benchmark, openapi, etc.)
    .env.example                    # Environment template with all 150+ variables

    app/
```

```
      main.py                       # FastAPI app factory, lifespan, ProductionMiddleware, routing
      api/
        dependencies.py             # Dependency injection factories (get_db, get_redis, get_qdrant)
        exception_handlers.py       # Global domain exception to HTTP response mapping
        v1/                         # 10 API Routers (auth, chat, documents, search, evaluation, health, etc.)
      auth/                         # Authentication, Argon2id, JWT, OAuth, session management, DEK crypto
      chat/                         # Orchestration, LLM history rewriter, prompt builder, Multi-Agent DAG
      documents/                    # Ingestion, extractors, chunking, embeddings, hybrid retrieval, graph
      core/
        config.py                   # 150+ Pydantic settings
        pagination.py               # Cryptographic CursorSigner, CursorPage[T], CursorDirection, HMAC-SHA256 tokens
        startup.py                  # Lifespan startup hooks (admin seeding, unverified cleanup, 2PC recovery)
        prompts/                    # System prompt templates (system_prompt.txt)
        cost/                       # YAMLPricingProvider, token cost calculator & pricing.yaml
        context/                    # RequestContext dataclass and ContextVar propagation
        email/                      # ResendEmailService with HTML templates & OTP generators
        feature_flags/              # Dynamic runtime feature flag engine (flags.py)
        logger.py                   # Structured logging, JSON formatting, PII maskers
        security/                   # Envelope encryption, ClamAV malware, prompt injection, quotas, rate_limiter
        reliability/                # Bulkheads, Circuit Breakers, Exponential Retries, Timeouts
        cache/                      # L1 Memory + L2 Redis Multi-Tier Cache, Repositories Caching, Mutex
        events/                     # Transactional Outbox Dispatcher, Redis Event Bus & PubSub
        queue/                      # Redis Stream Task Queue, JobDispatcher, Handlers & QueueWorker
        telemetry/                  # OpenTelemetry setup, Prometheus metrics, TelemetrySanitizer
        math/                       # Deterministic MathEngine, FinancialMath, Statistical, LaTeX, AST Sandbox
      telemetry/                    # Root telemetry package alias
      db/                           # Async SQLAlchemy sessionmaker, ORM Base, and 25 Model mappings
      evaluation/                   # 6 Evaluators, 5 Runners, 33 Metrics, Golden Datasets, Regression Gate
      rag/orchestration/            # Dedicated Adaptive RAG Framework (master, strategy, replanner, telemetry)

  frontend/
    src/
      main.tsx                      # React root render
      App.tsx                       # Client-side routing and context provider tree
      App.css                       # Application shell styling
      index.css                     # Global Vanilla CSS design tokens (light/dark mode)
      shiki-langs.d.ts              # Shiki language bundle typings
      workers/
        pyodide.worker.ts           # Pyodide WebAssembly Web Worker for sandboxed Python/Pandas
      lib/
        pyodide/pyodideManager.ts   # Web Worker RPC bridge and execution manager
        uploadStages.ts             # 7-stage progressive upload pipeline
        passwordStrength.ts         # Client-side password validation
        analytics.ts                # Client-side event tracking
        notifications.ts            # Browser desktop notifications
        highlighter.ts              # Shiki syntax highlighter
        constants.ts                # Global UI limits, polling timeouts, SSE event types
        messages.ts                 # Citation parsing and message content formatters
        utils.ts                    # Style combining (cn) and formatting helpers
      types/
        index.ts                    # Core TypeScript interfaces (Conversation, Message, Document, Citation, etc.)
      services/
        api.ts, auth.ts             # Abstracted frontend API communication services
      utils/
        lazyWithRetry.ts            # Resilient dynamic chunk loading wrapper
      api/                          # Axios client with interceptors, token refresh buffer & endpoints
      routes/                       # Route guards (AdminRoute, ProtectedRoute, PublicRoute)
      components/                   # ChatWindow, SSEStream, DocumentManager, CitationDrawer, CommandPalette, UI
      pages/                        # LandingPage, ProjectsPage, DocumentsPage, SettingsPage, ChatPage, etc.
      context/                      # AuthContext, ThemeContext, UploadQueueContext
      hooks/                        # useAuth, useConversations, useDocuments, useSSEChat, useDrafts, etc.
    e2e/                            # Playwright automated test suites (login, delete, two-tier, redis)
```

4 Part II: End-to-End Sequence Diagrams & Step-by-Step Data Flows

4.1 5. Document Ingestion, Parsing, Chunking & Vector Indexing Sequence

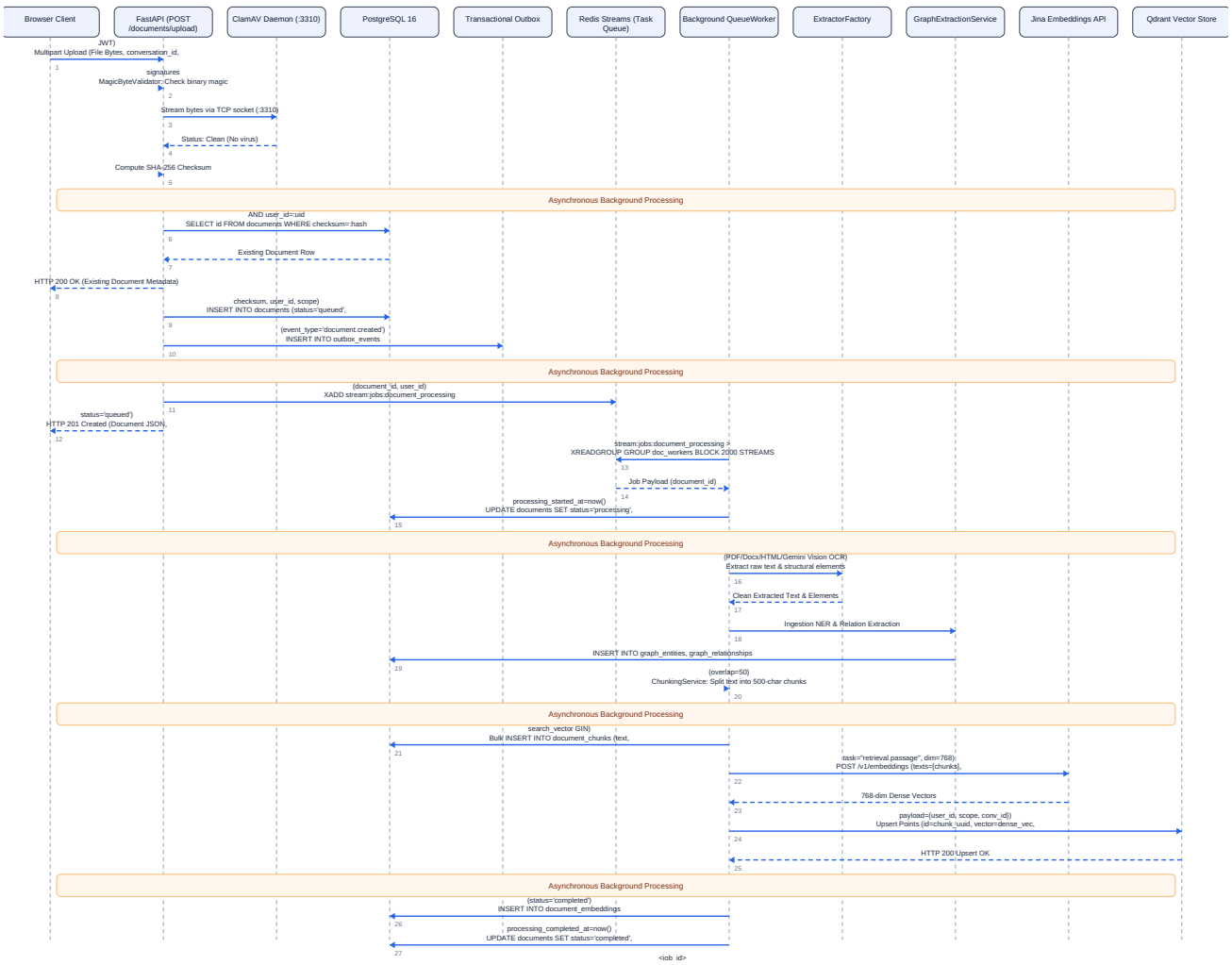


Figure 1 — rendered from the Mermaid diagram in the source document.

4.2 6. Real-Time Chat & SSE Streaming Lifecycle Sequence

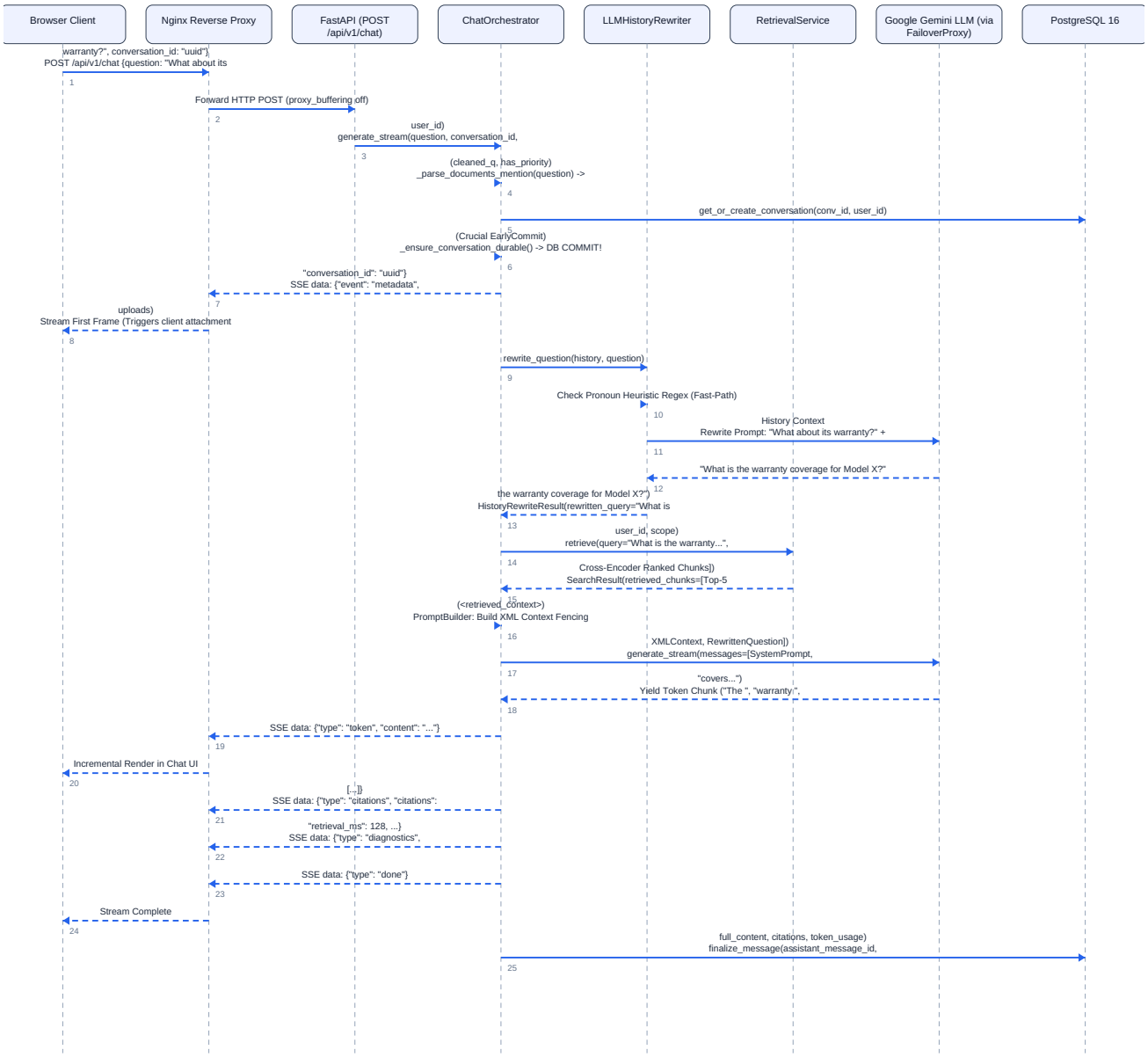


Figure 2 — rendered from the Mermaid diagram in the source document.

4.3 7. Production Hybrid Retrieval Pipeline Sequence (Dense + Sparse + RRF + MMR + Reranker)

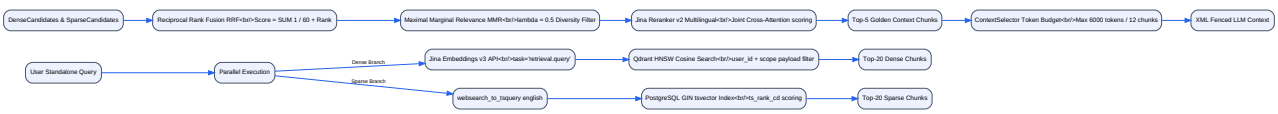


Figure 3 — rendered from the Mermaid diagram in the source document.

4.4 8. Hierarchical Multi-Agent DAG Planning & Execution Wave Sequence

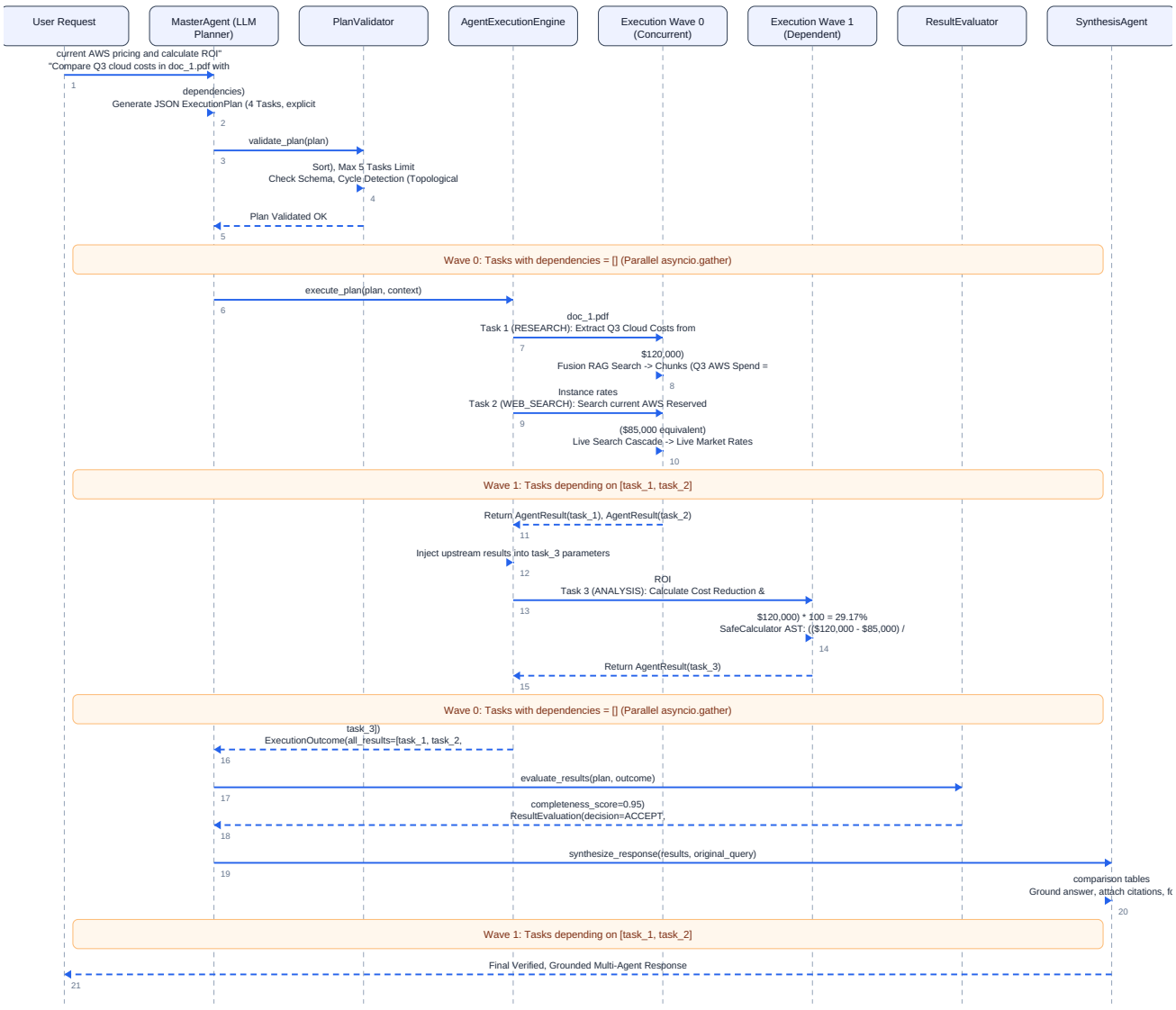


Figure 4 — rendered from the Mermaid diagram in the source document.

4.5 9. Authentication, Token Rotation, CSRF & Replay Attack Defense Sequence

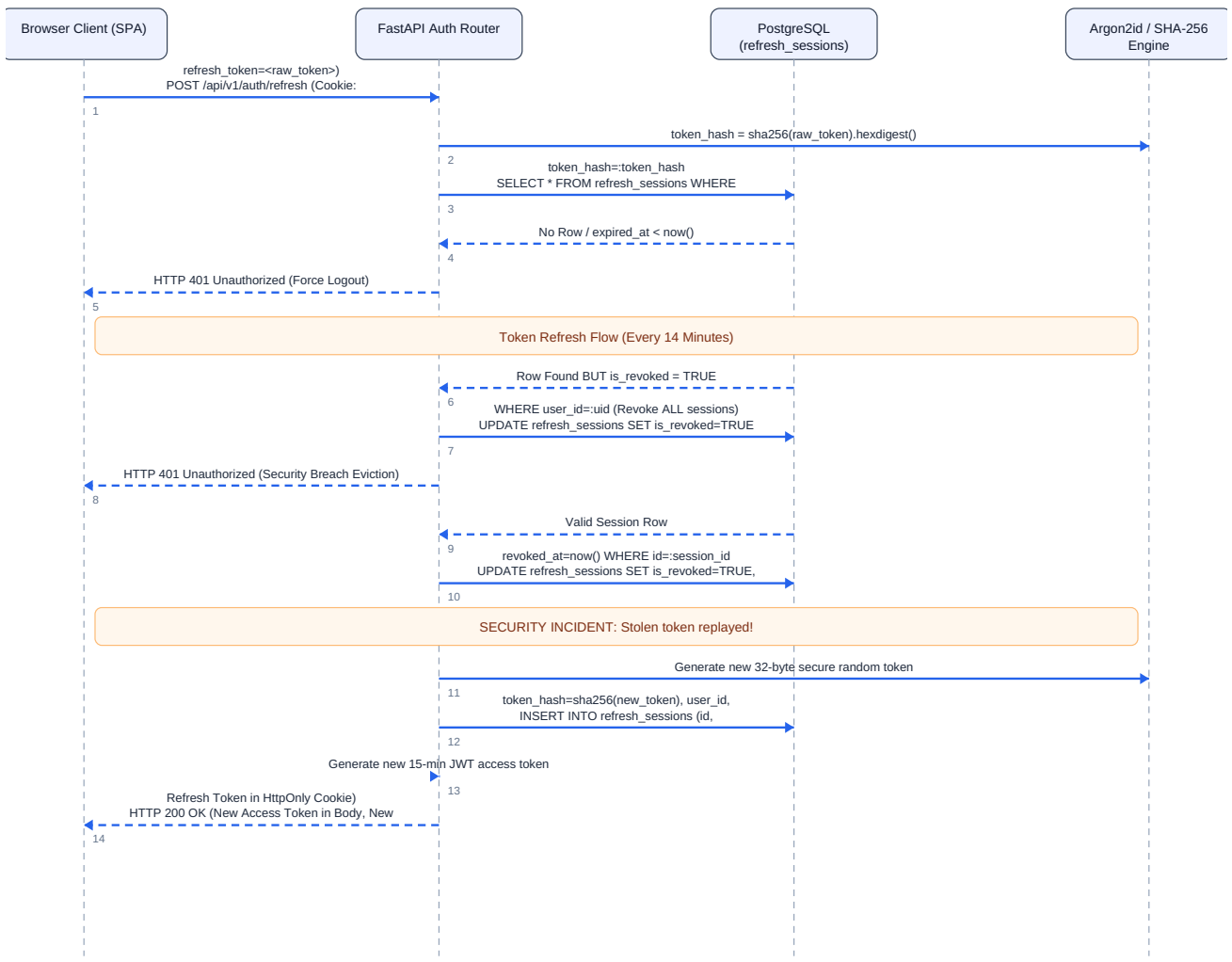


Figure 5 — rendered from the Mermaid diagram in the source document.

4.6 10. Transactional Outbox Pattern & Event Bus Sequence

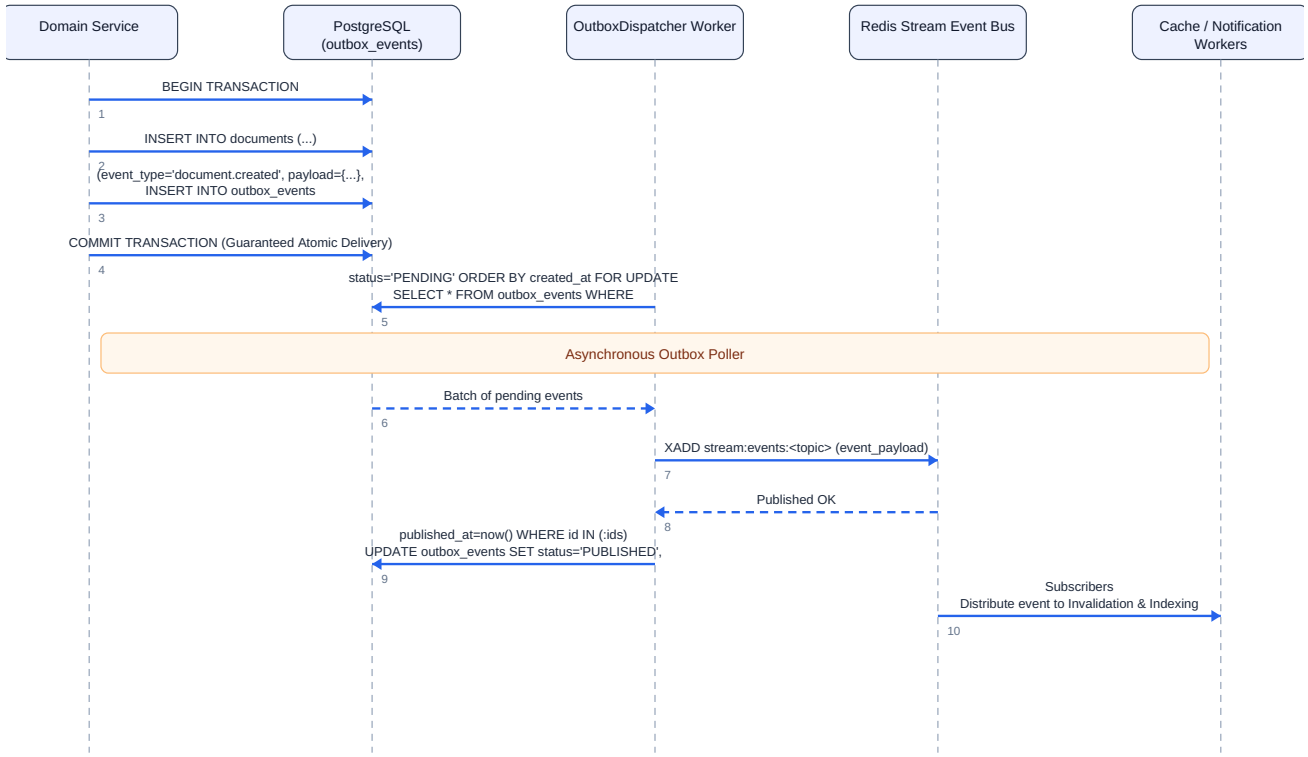


Figure 6 — rendered from the Mermaid diagram in the source document.

4.7 11. Knowledge Graph Extraction & Multi-Hop Traversal Sequence

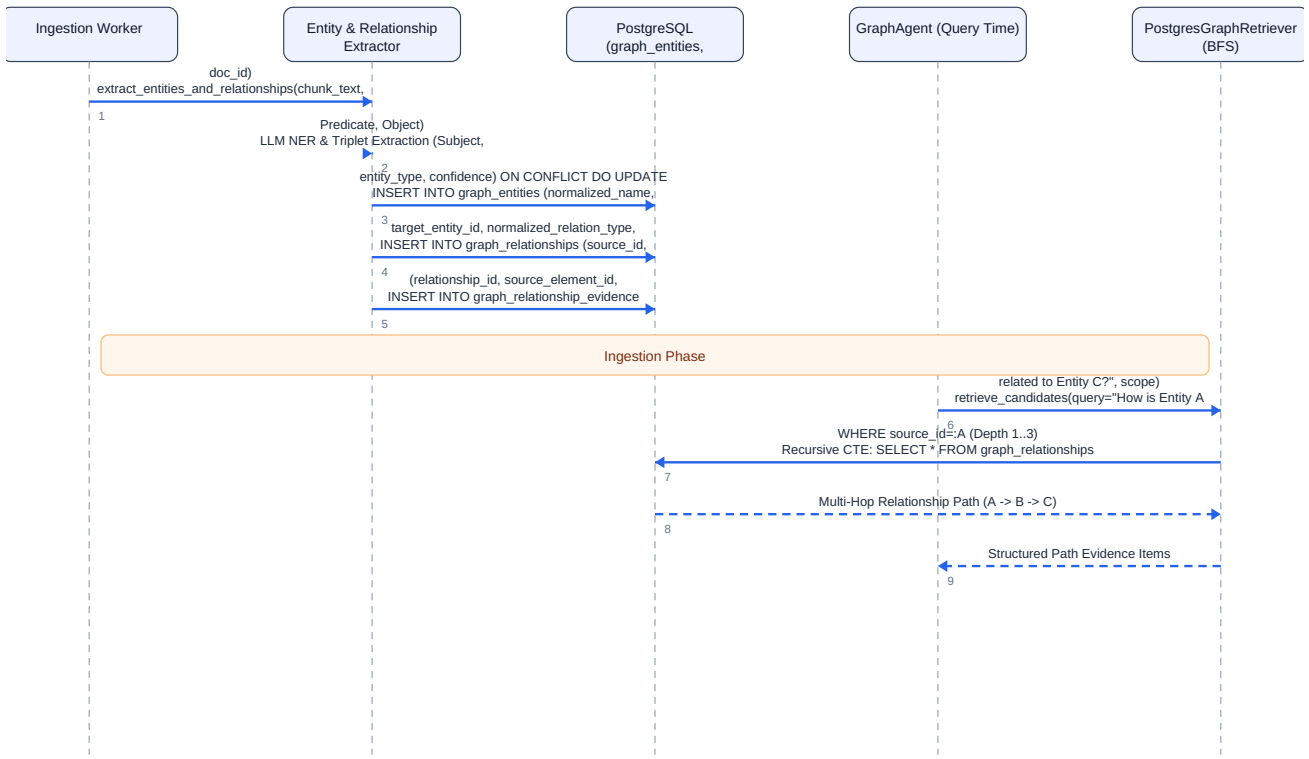


Figure 7 — rendered from the Mermaid diagram in the source document.

5 Part III: Specialist Agent, Tool Calling & Dual-Layer Orchestration Architecture

5.1 12. Dual Orchestration Layers: Adaptive RAG vs. Hierarchical Multi-Agent DAG

The platform distinguishes between two separate orchestration layers depending on request complexity:

1. LIGHTWEIGHT ADAPTIVE RAG LAYER (app/rag/orchestration/)

Purpose: Sub-second, low-overhead retrieval strategy routing for standard user queries.
Components: AdaptiveRAGOrchestrator, RetrievalStrategyRouter, EvidenceSufficiencyEvaluator.
Strategies: DIRECT (Single-pass), HYDE (Hypothetical), MULTI_HOP (Sub-query), SPECIALIST.

→

2. FULL HIERARCHICAL MULTI-AGENT DAG LAYER (app/chat/services/multi_agent/)

Purpose: Multi-domain reasoning, AST math computation, live web searching, tabular/graph.
Components: MasterAgent, PlanValidator, AgentExecutionEngine, ResultEvaluator, 7 Specialists, SynthesisAgent, TaskRouter, CapabilityRegistry, and single-agent Tool Calling loop.

```
================================================================================================================================
                                    MULTI-AGENT & SPECIALIST AGENT ARCHITECTURE MATRIX
================================================================================================================================
Component / Agent   File Location                                          Primary Role & Capabilities
--------------------------------------------------------------------------------------------------------------------------------
-- 1. Orchestrator, Planners & Verification Infrastructure --
1. MasterAgent      app/chat/services/multi_agent/master_agent.py      LLM DAG Planner, Bounded Replanner, Domain Context Gate
2. ExecutionEngine  app/chat/services/multi_agent/execution_engine.py  Topological Wave Executor, Async Sorter, Timeout Guard
3. PlanValidator    app/chat/services/multi_agent/plan_validator.py    Schema Validator, Topological Cycle Detector, Whitelist Guard
4. ResultEvaluator app/chat/services/multi_agent/result_evaluator.py  LLM-as-Judge Result Scoring (0.0-1.0), Accept/Replan Decisions
5. TaskRouter       app/chat/services/multi_agent/task_router.py       Capability Matching, Cost/Latency Constraint Router
6. CapabilityReg    app/chat/services/multi_agent/capability_registry.py Domain Definitions, Tool Whitelists, Token Limit Budgets
7. AgentExecutor    app/chat/services/agent_executor.py                Single-Agent Function Calling Loop (7 Registered Tools)

-- 2. Seven Core Specialist Execution Workers --
8. ResearchAgent    app/chat/services/multi_agent/research_agent.py    Decomposed Sub-Query Search, Hybrid RAG, Provenance Deduplication
9. ResearchPlanner  app/chat/services/multi_agent/research_planner.py  Pure Sub-Query DAG Planner (Zero I/O)
10. DocumentAgent   app/chat/services/multi_agent/document_agent.py    UUID Inspection, Chunk Lookup, Scoped Document Listing
11. AnalysisAgent   app/chat/services/multi_agent/analysis_agent.py    Deterministic SafeCalculator AST, SymPy MathEngine, Python Scripts
12. WebSearchAgent  app/chat/services/multi_agent/web_search_agent.py  SerpAPI -> Firecrawl -> DuckDuckGo Cascade, Snippet Extractor
13. TableAgent      app/chat/services/multi_agent/table_agent.py       TableProcessor, Markdown/CSV Parsers, Column Aggregations (SUM/AVG)
14. VisionAgent     app/chat/services/multi_agent/vision_agent.py      VisionProcessor, Multimodal Diagram/Chart OCR via Gemini Vision
15. GraphAgent      app/chat/services/multi_agent/graph_agent.py       PostgresGraphRetriever, Entity Relationship Traversal, Multi-Hop

-- 3. Grounding & Response Synthesis --
16. SynthesisAgent  app/chat/services/multi_agent/synthesis_agent.py   Evidence Grounding, Refusal Faithfulness, Citation Tagging
================================================================================================================================
```

5.2 13. MasterAgent: Hierarchical Planner, Replanner & Evidence Gating

Located in master_agent.py:

5.2.1 13.1 Dynamic DAG Planning & Prompt Engineering

The `MasterAgent` serves as the centralized planning coordinator. It operates under `PLANNER_SYSTEM_PROMPT` which dynamically instructs the LLM to formulate an optimal directed acyclic graph (DAG) of discrete tasks for any user request:

- **Allowed Specialist Types**: `RESEARCH`, `DOCUMENT`, `ANALYSIS`, `WEB_SEARCH`, `TABLE`, `VISION`, `GRAPH`.

- **Structured JSON Schema**:

```
{
  "reasoning_summary": "Explanation of multi-step breakdown and dependencies",
  "tasks": [
    {
      "task_id": "task_1",
      "agent_type": "RESEARCH",
      "request": "Extract relevant text chunks from uploaded document",
      "dependencies": [],
      "parameters": {"query": "annual revenue 2022 to 2024", "top_k": 5}
    },
    {
      "task_id": "task_2",
      "agent_type": "TABLE",
      "request": "Extract financial revenue table and compute row metrics",
      "dependencies": ["task_1"],
      "parameters": {"operation": "GROWTH"}
    },
    {
      "task_id": "task_3",
      "agent_type": "ANALYSIS",
      "request": "Calculate exact CAGR percentage between first and last values",
      "dependencies": ["task_2"],
      "parameters": {"action": "calculate"}
    }
  ]
}
```

5.2.2 13.2 Deterministic Heuristic Fallback Planner (`_plan_heuristic`)

If the LLM provider fails (e.g. HTTP 429 quota exhaustion or timeout), `MasterAgent` immediately falls back to `_plan_heuristic`:

- **Table Intent**: Detects keywords (`"table"`, `"csv"`, `"tabular"`, `"rows"`, `"columns"`) and chains `RESEARCH -> TABLE -> (optional ANALYSIS)`.
- **Vision Intent**: Detects keywords (`"image"`, `"chart"`, `"diagram"`, `"figure"`, `"visual"`, `"plot"`) and plans `VISION` + `RESEARCH` in parallel.
- **Graph Intent**: Detects keywords (`"related to"`, `"relationship"`, `"connection between"`, `"knowledge graph"`, `"lineage"`) and plans `GRAPH` + `RESEARCH`.
- **Web Search Intent**: Detects keywords (`"online"`, `"latest"`, `"news"`, `"current price"`, `"market data"`) and plans `RESEARCH` + `WEB_SEARCH` concurrently feeding into `ANALYSIS`.
- **Pure Math Intent**: Detects arithmetic expressions and directly routes to `ANALYSIS`.

5.2.3 13.3 Multi-Agent Internal Pydantic Contracts

Located in multi_agent.py:

- **`AgentTask`**: `{ task_id: str, agent_type: AgentType, request: str, dependencies: list[str], parameters: dict[str, Any], priority: int, expected_outputs: list[str] }`
- **`ExecutionPlan`**: `{ request_id: str, tasks: list[AgentTask], reasoning_summary: str, iteration: int }`
- **`EvidenceItem`**: `{ chunk_id: UUID | None, document_id: UUID | None, document_name: str | None, content: str, score: float | None, metadata: dict[str, Any] }`
- **`AgentResult`**: `{ task_id: str, agent_type: AgentType, status: AgentTaskStatus, success: bool, output: str, evidence: list[EvidenceItem], citations: list[Citation], execution_time_ms: float }`
- **`MultiAgentResponse`**: `{ request_id: str, answer: str, citations: list[Citation], specialist_results: list[AgentResult], plan_iterations: int, total_duration_ms: float }`

5.2.4 13.4 Evidence Relevance Gate Algorithm

```
def _apply_evidence_relevance_gate(
    self, results: list[AgentResult], context: AgentExecutionContext
) -> tuple[list[AgentResult], int]:
    filtered_results = []
    rejected_count = 0
    for res in results:
        # Validate tenant boundary and domain relevance
        valid_evidence = [
            item for item in res.evidence
            if item.content and not self._is_off_domain_or_malicious(item.content, context)
        ]
        rejected_count += len(res.evidence) - len(valid_evidence)
        res.evidence = valid_evidence
        filtered_results.append(res)
    return filtered_results, rejected_count
```

5.3 14. AgentExecutionEngine: Topological Wave Executor & Concurrency Controller

Located in execution_engine.py:

1. Calculates in-degrees for all tasks based on their `dependencies: list[str]`.
2. Groups independent tasks (in-degree = 0) into **Wave 0**.
3. Executes Wave 0 tasks concurrently via `asyncio.gather()` (4.00× measured parallel speedup across 4 tasks).
4. Injects completed outputs into dependent tasks in **Wave 1** via `context.dependency_results[dep_id]` and executes the next wave.
5. Enforces per-task timeouts (`MULTI_AGENT_TASK_TIMEOUT=20.0s`) using `asyncio.wait_for()`.
6. Fault Isolation: If an upstream required task fails, dependent downstream tasks are gracefully skipped with `AgentFailureReason.DEPENDENCY_FAILED`.

5.4 15. Dedicated Adaptive RAG Orchestration Framework (`app/rag/orchestration/`)

Located in `backend/app/rag/orchestration/`:

- **`AdaptiveRAGOrchestrator` (`master.py`)**: Master state machine managing query lifecycle, task budgeting, and worker dispatching across 4 core strategies.
- **`EvidenceSufficiencyEvaluator` (`evidence_evaluator.py`)**:
  - Computes composite confidence metric $S_{\text{evidence}} \in [0.0, 1.0]$ based on reranker score distribution, cross-chunk agreement, and coverage density.
  - Gating threshold: $S_{\text{evidence}} \geq 0.70$ triggers direct LLM answer synthesis; $S_{\text{evidence}} < 0.70$ triggers replanner widening.
- **`RetrievalStrategyRouter` (`strategy_router.py`)**: Rule-based & semantic classifier selecting between `DIRECT`, `HYDE`, `MULTI_HOP`, or `SPECIALIST`.
- **`Replanner` (`replanner.py`) & `compute_task_fingerprint`**: Generates task delta plans. Uses SHA-256 fingerprinting on normalized sub-query strings and scope filters to guarantee zero duplicate retrieval runs.
- **`OrchestrationTelemetry` (`telemetry.py`)**: Tracks granular event logs (`OrchestrationTelemetryEvent`) for Jaeger spans and frontend execution cards.
- **`RetrievalWorker` (`worker.py`)**: Non-blocking worker coroutine executing isolated retrieval tasks.

5.5 16. ResearchAgent & ResearchPlanner: Sub-Query Decomposition & KB Retrieval

Located in research_agent.py:

- **Decomposition**: Uses `ResearchPlanner` to break complex research topics into 1-4 targeted sub-queries.
- **Deduplication**: Computes cosine similarity across sub-query embeddings; discards queries with similarity > 0.85 to avoid redundant retrievals.
- **Hybrid Retrieval**: Executes full Hybrid RAG (Qdrant Dense + PostgreSQL FTS + RRF + MMR + Reranker) for each sub-query.
- **Conflicting Evidence Detection**: Applies heuristics and semantic checks (`_detect_conflicting_evidence()`) to flag contradictory claims across retrieved chunks.

5.6 17. DocumentAgent: Explicit Document & Chunk Inspection Specialist

Located in document_agent.py:

- **Document Scoping**: Lists available documents filtered by `user_id` and active `conversation_id`.
- **Direct UUID Fetching**: Looks up exact document chunks and structural metadata directly from PostgreSQL by UUID.
- **Authorization Fencing**: Rejects requests attempting to access documents outside the user's authorized scope.

5.7 18. AnalysisAgent: Deterministic Math, AST Sandboxing, SymPy & MathEngine

Located in analysis_agent.py and engine.py:

- **Upstream Metric Extraction**: Automatically extracts currency figures, numbers, and percentage deltas from upstream `context.dependency_results`.
- **`FinancialMathSubsystem`**:
  - `cagr(beginning_value, ending_value, periods)`: Compound annual growth rate calculation.
  - `net_present_value(discount_rate, cash_flows)`: NPV derivation across series.
  - `internal_rate_of_return(cash_flows)`: Exact IRR computation via Newton-Raphson approximation.
  - `loan_amortization(principal, rate, periods)`: Complete monthly principal and interest schedule.
  - `wacc(equity, debt, cost_of_equity, cost_of_debt, tax_rate)`: Weighted average cost of capital.
- **`StatisticalNumericalSubsystem`**:
  - Descriptive statistics (mean, median, variance, standard deviation, skewness, kurtosis).
  - Hypothesis testing (two-sample t-test, one-way ANOVA, Mann-Whitney U rank test).
  - Curve fitting (`curve_fit` for linear, polynomial, logarithmic, and exponential regressions).
- **`LaTeXStepFormatter`**: Emits step-by-step mathematical derivations formatted in KaTeX-compatible LaTeX expressions.

- **AST SafeCalculator**: AST expression parser validating nodes against whitelisted math operations (`abs`, `round`, `sum`, `min`, `max`, `pow`).

5.8 19. WebSearchAgent: Real-Time Multi-Provider Web Search Cascade

Located in web_search_agent.py:

- **Automated Failover Cascade**: Tries primary provider **SerpAPI** → falls back to **Firecrawl** → falls back to **DuckDuckGo**.
- **Provenance & Snippets**: Extracts top 5 web results, formats them with title, URL, and snippet text, and generates deterministic UUIDv5 `EvidenceItem` records.

5.9 20. TableAgent: Tabular Data Extraction & Column Aggregations

Located in table_agent.py:

- **Triple-Source Ingestion**:
  1. Task parameters (`table_markdown` or `table_csv`).
  2. Upstream DAG dependency results (`context.dependency_results` from `ResearchAgent`).
  3. `DocumentElementRepository.list_by_version(user_id, doc_id, version)` for persisted table elements.
- **Structured Data Model (`StructuredTable`)**:
  - Parses raw markdown/CSV tables into `StructuredTable` containing `TableDimension` and `cells: list[TableCell]`.
  - Sanitizes cell values, stripping non-numeric formatting (e.g. `"$100M"` → `100.0`).
- **Deterministic Aggregations**:
  - `GROWTH`: Calculates exact percentage growth between first and last numeric cell values: $ \text{Growth} = \frac{v_{\text{last}} - v_{\text{first}}}{v_{\text{first}}} \times 100\%$.
  - Column operations: `SUM`, `AVG`, `MIN`, `MAX`, `COUNT`, `CAGR`.
- **Structured Evidence**: Emits `EvidenceItem` with `document_name="Structured Table"`, column headers, dimensions (`row_count x col_count`), and markdown summary.

5.10 21. VisionAgent: Multimodal Visual Reasoning & Chart Inspection

Located in vision_agent.py:

- **Multi-Source Image Ingestion**: Accepts raw `image_bytes`, base64-encoded strings (`image_bytes_b64`), or queries persisted visual elements (`ElementType.FIGURE`, `ElementType.CHART`) from `DocumentElementRepository`.
- **Visual Analysis Engine**: Interfaces with `VisionProcessor` backed by Gemini Vision API to generate structural layout breakdowns, OCR transcriptions, and chart trend analyses.
- **Structured Evidence**: Emits `EvidenceItem` with `document_name=caption or f"Visual {element_type}"`, caption metadata, and extracted data points.

5.11 22. GraphAgent: Knowledge Graph Traversal & Entity Relationship Reasoning

Located in graph_agent.py:

- **Multi-Hop Traversal**: Traverses relational knowledge graph edges via `PostgresGraphRetriever` and `GraphTraversalService`.
- **Active Library Document Auto-Discovery**: If specific document IDs are omitted, automatically queries the user's active document versions from PostgreSQL.
- **Entity Relationship Reasoning**: Extracts relational triples `(Entity A) --[RELATION_TYPE, confidence]--> (Entity B)` and attaches direct source chunk snippets (`RelationshipEvidence`).
- **Strict No-Hallucination Fallback**: If no relationships exist between queried entities in the user's graph, returns 0 evidence items rather than fabricating connections.

5.12 23. SynthesisAgent: Context Grounding, Citation Tagging & Refusal Faithfulness

Located in synthesis_agent.py:

- **Strict Evidence Integrity**: Constructs responses using **only** successful specialist results and verified `EvidenceItem` chunks.
- **Refusal Faithfulness**: Enforces explicit refusal statements when evidence is missing, conflicting, or inconclusive for factual queries.
- **Citation Attribution**: Maps facts to inline document citations with source titles, URLs, and page numbers.
- **Untrusted Context Fencing**: Treats all evidence as untrusted external data enclosed in `<retrieved_context>` XML tags.

5.13 24. ResultEvaluator: Task Quality Scoring & Adaptive Replanning Engine

Located in result_evaluator.py:

- **Completeness & Evidence Scoring**: Evaluates completed task outputs against `expected_outputs` and `required_evidence`, generating a score between 0.0 and 1.0.
- **Decision Engine**: Emits `ACCEPT` (score ≥ 0.85), `REPLAN` (score < 0.85 and replan budget remains), or `FAIL`.

5.14 25. PlanValidator: Schema, Cycle Detection & Constraint Enforcement

Located in plan_validator.py:

- **Schema Validation**: Guarantees all required fields (`task_id`, `agent_type`, `request`, `dependencies`) exist and are non-empty.
- **Topological Cycle Detection (Kahn's Algorithm)**: Computes node in-degrees and verifies that the execution graph is a valid DAG. Rejects circular dependencies ($A \rightarrow B \rightarrow A$) with `['Circular dependency detected in execution plan.']`.

- **Dependency Existence**: Rejects tasks that reference nonexistent dependency IDs or declare self-dependencies.
- **Task Limit Enforcement**: Enforces `MAX_SPECIALIST_TASKS = 5` to prevent runaway LLM costs.

5.15 26. TaskRouter & CapabilityRegistry: Capability-Aware Cost/Latency Routing

Located in task_router.py and capability_registry.py:

- Maintains a registry of 7 core agent capabilities:
  1. `research.fusion_rag` (`ResearchAgent`)
  2. `document.direct_inspection` (`DocumentAgent`)
  3. `analysis.math_engine` (`AnalysisAgent`)
  4. `web.realtime_search` (`WebSearchAgent`)
  5. `table.structured_engine` (`TableAgent`)
  6. `vision.multimodal_inspection` (`VisionAgent`)
  7. `graph.knowledge_traversal` (`GraphAgent`)
- Evaluates estimated latency and cost units against task limits before dispatching.

5.16 27. Tool Calling, Function Declaration Schemas & Discovery Engine

Located in `backend/app/chat/services/`:

- **`ToolDiscoveryService` (`tool_discovery.py`)**:
  - Reflection engine that converts Python typing annotations and Pydantic schemas into Google Gemini Function Declaration JSON formats (`name`, `description`, `parameters.type`, `required`, `properties`).
- **`ToolExecutor` (`tool_executor.py`)**:
  - Sandboxed execution with strict timeouts (`TOOL_EXECUTION_TIMEOUT=30.0s`), result payload size truncation (`TOOL_RESULT_SIZE_LIMIT=50000` chars), automatic 1-step retry on JSON parsing errors (`TOOL_AUTO_RETRY_COUNT=1`), and recursion loops protection (`TOOL_RECURSION_PROTECTION=True`).
- **`ToolRegistry` (`tool_registry.py`)**:
  - Case-insensitive tool lookup and alias mapping engine (`register(tool, aliases=['calc', 'math'])`).
- **7 Core Registered Tools (`app/chat/services/tools/`)**:
  1. `SafeCalculator`: AST arithmetic evaluator (`expression: str`).
  2. `WebSearchTool`: Live internet search (`query: str, max_results: int = 5`).
  3. `SemanticSearchTool`: Qdrant dense vector search (`query: str, limit: int = 10`).
  4. `ChunkRetrievalTool`: Direct chunk fetch by UUID (`chunk_id: UUID`).
  5. `DocumentRetrievalTool`: Document metadata fetch (`document_id: UUID`).
  6. `CitationLookupTool`: Source verification (`citation_id: str`).
  7. `DateTimeTool`: Current UTC/local timestamps (`timezone: str = "UTC"`).

5.17 28. Multi-Agent & Chat Operational Subsystems: Keyset Pagination, Memory & LLM Failover

- **Cryptographic Keyset / Cursor-Based Pagination (`app/core/pagination.py`)**:
  - Replaces fragile, high-latency `OFFSET / LIMIT` pagination with tamper-evident HMAC-SHA256 signed opaque cursor tokens (`CursorSigner`).
  - **Wire Token Format**: `base64url(canonical_json) + "." + base64url(hmac_sha256(canonical_json, CURSOR_SIGNING_KEY))`.
  - **Payload Structure (`CursorPayload`)**: Encodes `{ v: 1, ts: "<ISO-8601 UTC>", id: "<UUID>", dir: "next"|"prev", res: "conv"|"msg", ctx: "<user_id|conv_id>", iat: <int>, exp: <int> }` with TTL validation (`CURSOR_TTL_SECONDS=604800` / 7 days).
  - **Bidirectional Row-Value Tuple Comparisons**:
    - Descending / Next: `tuple_(created_at, id) < tuple_(literal(cursor_ts), literal(cursor_id))` with `ORDER BY created_at DESC, id DESC LIMIT :limit + 1`.
    - Ascending / Prev: `tuple_(created_at, id) > tuple_(literal(cursor_ts), literal(cursor_id))` with `ORDER BY created_at ASC, id ASC LIMIT :limit + 1` (results re-reversed in memory).
  - **Envelope Model (`CursorPage[T]`)**: Standardized pagination envelope exposing `items: list[T]`, `limit: int`, `next_cursor: str | None`, `prev_cursor: str | None`, `has_next_page: bool`, `has_prev_page: bool`, `has_more: bool`.
- **`TokenEstimator` (`token_estimator.py`)**:
  - Pre-flight prompt assembly validation and multi-modal token budgeting prior to LLM dispatch.
  - Calculates accurate token bounds across:
    - Text: Character-to-BPE approximation with special framing tokens (`<|im_start|>`, `<|im_end|>`).
    - Tool Definitions: Gemini Function Declaration JSON schema serialization footprint.
    - Citations: Formatted XML/Markdown bracket markers and source metadata overhead.
    - Vision Images: Multi-tile bounding box formula ($N = \lceil width/512 \rceil \times \lceil height/512 \rceil \times 258$ tokens).
- **`HistoryRewriter` (`history_rewriter.py`)**:
  - Coreference and pronoun resolution engine transforming conversational follow-up questions (e.g., *"What were its total assets in 2024?"*) into self-contained standalone search queries prior to hybrid retrieval.
  - Leverages rolling conversation summaries and recent message turns to resolve ambiguous entity mentions without polluting vector search queries.
- **`ProviderFailoverProxy` (`app/chat/infrastructure/failover_proxy.py`)**: Subclasses `LLMProviderRouter` with circuit breaker integration. Intercepts `429 Too Many Requests` or `5xx Server Error` on primary `GEMINI_API_KEY` and transparently fails over to `GEMINI_FALLBACK_API_KEY` without breaking active client streams.
- **`ConversationAccessService` (`access_service.py`)**: Centralized ownership & authorization verification protecting conversations and scoped document attachments.
- **`ConversationSummarizer` & `RollingMemoryManager` (`conversation_summarizer.py`, `rolling_memory.py`)**: Triggered when active conversation length exceeds `SUMMARY_TRIGGER_MESSAGES=20`, producing incremental records in `conversation_summaries`.
- **Message Branching & Edit Truncation (`delete_messages_from`)**: ChatGPT-style message editing where deleting or modifying a message cascades and deletes all subsequent replies.
- **`AnswerGenerationService` (`answer_generation_service.py`)**: Non-streaming answer generator with citation grounding.

- **Multi-Agent Health Diagnostics (`health_diagnostics.py`)**: Monitors background specialist health and marks degraded agents for automatic bypass.
- **Automated Title Generation (`title_generator.py`)**: Non-blocking background extraction and LLM summarization for conversation titles.
- **Conversation Retention Service (`retention_service.py`)**: Automated sweeps archiving conversations older than `MAX_CONVERSATION_AGE_DAYS=90`.

---

6 Part IV: Relational Database Schema, Data Models & Repository Layer (All 25 Tables & 14 Repositories)

6.1 29. Relational Architecture, Entity-Relationship Diagram (ERD) & Bounded Contexts

6.1.1 29.1 Composite Foreign Keys, Partial Indexes & SQLite Variants

- **Composite Tenant Fencing**: Tables (`document_processing_runs`, `document_processing_jobs`, `document_elements`, `graph_relationship_evidence`, `graph_entity_mentions`) enforce composite FKs on `["user_id", "document_id"]` referencing `documents(user_id, id)` with `ON DELETE CASCADE`.
- **Partial Unique Index**: `CREATE UNIQUE INDEX uq_processing_runs_doc_active ON document_processing_runs(document_id) WHERE state = 'ACTIVE';` ensures a single active pipeline run per document.
- **Table Constraints**:
  - `documents`: `uq_documents_user_checksum (user_id, checksum)`, `ck_documents_active_version_nonneg (active_version >= 0)`, `ck_documents_version_ge_active (version >= active_version)`.
  - `outbox_events`: `uq_outbox_event_idempotency (user_id, document_id, document_version, event_type)`.
  - `durable_cleanup_jobs`: `uq_cleanup_resource (resource_type, resource_id)`.
- **Column Alias Mapping**: SQLAlchemy model attribute aliases map directly to raw DB column names:
  - `settings_json` → DB column `"settings"`
  - `metadata_json` → DB column `"metadata"`
  - `payload_json` → DB column `"payload"`
  - `properties_json` → DB column `"properties"`
  - `bounding_box_json` → DB column `"bounding_box"`
- **Cross-Dialect SQLite Type Variance**: Uses `.with_variant(JSON(), "sqlite")`, `.with_variant(Integer(), "sqlite")`, and `.with_variant(Text(), "sqlite")` across `JSONB`, `BigInteger`, and `TSVECTOR` columns for in-memory SQLite unit testing.

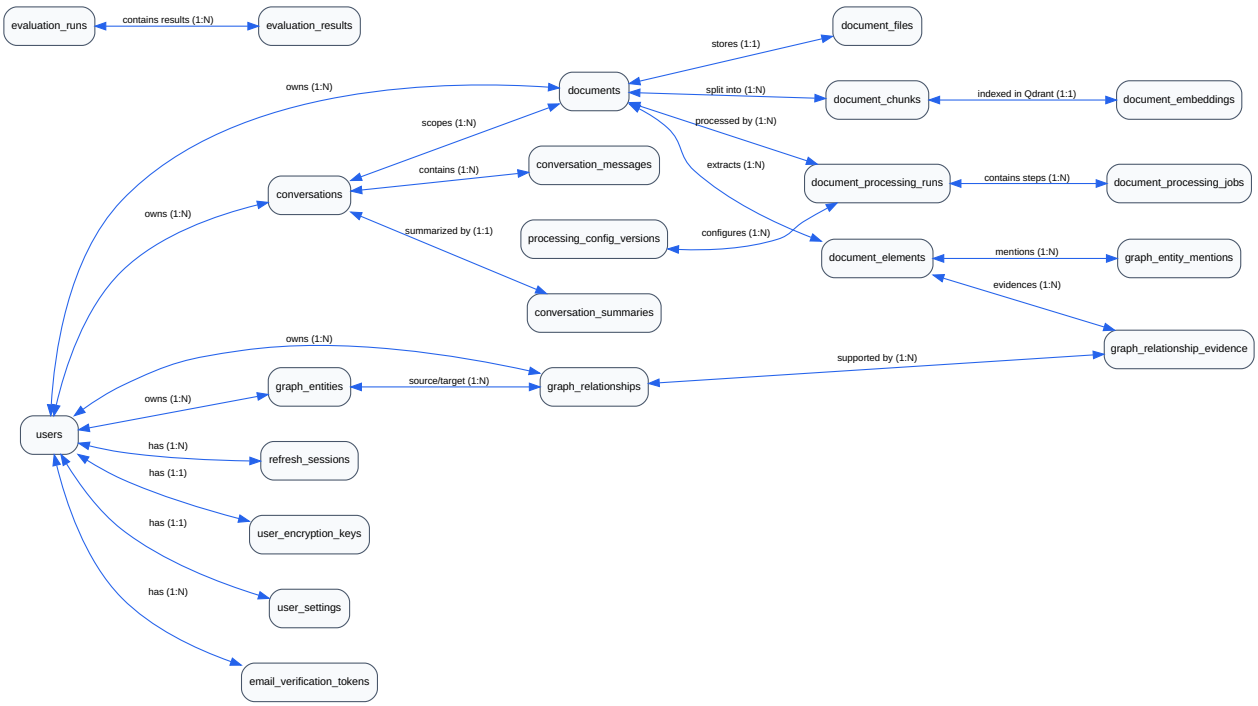


Figure 8 — rendered from the Mermaid diagram in the source document.

6.2 30. Domain Repository Architecture & Abstract Interfaces

The persistence layer follows Domain-Driven Design (DDD) separating abstract interfaces from asynchronous SQLAlchemy implementations:

```
================================================================================================================
                                    ALL 14 DOMAIN PERSISTENCE IMPLEMENTATIONS
================================================================================================================
Persistence / Repo Class     File Location                                                  Primary Responsibilities
----------------------------------------------------------------------------------------------------------------
1. SQLAlchemyUserRepository  app/auth/infrastructure/persistence/repositories/             User CRUD, OAuth sub lookup, verification
2. SQLAlchemySessionRepo     app/auth/infrastructure/persistence/repositories/             Token rotation, replay attack revocation
3. KeyManager (Crypto DEK)   app/core/security/key_manager.py                              Wrapped DEK persistence, envelope encryption
4. EmailVerificationTokenRepoapp/auth/infrastructure/persistence/repositories/             OTP verification, rate-limited attempts
5. UserSettingsService       app/auth/services/user_settings_service.py                    JSONB preferences, folders, filter presets
6. ConversationRepository    app/chat/infrastructure/persistence/repositories/             Threads, optimistic locking, archiving
7. MessageRepository         app/chat/infrastructure/persistence/repositories/             Message history, branch deletions
8. SummaryRepository         app/chat/infrastructure/persistence/repositories/             Incremental rolling summaries
9. DocumentRepository        app/documents/infrastructure/persistence/repositories/        Document metadata, scoping, deduplication
10. ChunkRepository          app/documents/infrastructure/persistence/repositories/        Text chunks, GIN full-text search
11. ProcessingRunRepository  app/documents/infrastructure/persistence/repositories/        State transitions (PENDING->ACTIVE->DONE)
12. ProcessingConfigRepo     app/documents/infrastructure/persistence/repositories/        Versioned chunking configs & hashing
13. DocumentElementRepo      app/documents/infrastructure/persistence/repositories/        Multimodal bounding boxes & spans
14. SQLAlchemyGraphRepo      app/documents/infrastructure/persistence/repositories/        Unified entities, relations, evidence & mentions
================================================================================================================
```

6.3 31. Authentication & Security Context (5 Tables & Repositories)

6.3.1 31.1 `users` Table

- `id` (UUID, Primary Key, default `gen_random_uuid()`)
- `email` (VARCHAR(255), Unique, Indexed)
- `hashed_password` (VARCHAR(255), Nullable for OAuth users)
- `role` (VARCHAR(50), default `'user'`) — `user`, `admin`
- `is_active` (BOOLEAN, default `TRUE`, Indexed)

- `is_verified` (BOOLEAN, default `FALSE`, Indexed)
- `oauth_provider` (VARCHAR(50), Nullable) — `google`, `github`
- `oauth_sub` (VARCHAR(255), Nullable, Indexed)
- `created_at` (TIMESTAMPTZ, default `now()`)
- `updated_at` (TIMESTAMPTZ, default `now()`)

6.3.2 31.2 `refresh_sessions` Table

- `id` (UUID, Primary Key)
- `user_id` (UUID, Foreign Key → `users.id` ON DELETE CASCADE, Indexed)
- `token_hash` (VARCHAR(64), Unique, Indexed) — SHA-256 hash of raw refresh token
- `ip_address` (VARCHAR(45))
- `user_agent` (TEXT)
- `is_revoked` (BOOLEAN, default `FALSE`, Indexed)
- `revoked_at` (TIMESTAMPTZ, Nullable)
- `last_used_at` (TIMESTAMPTZ, default `now()`)
- `expires_at` (TIMESTAMPTZ, Indexed)
- `created_at` (TIMESTAMPTZ, default `now()`)

6.3.3 31.3 `user_encryption_keys` Table

- `user_id` (UUID, Primary Key, Foreign Key → `users.id` ON DELETE CASCADE)
- `wrapped_dek` (TEXT) — AES-256-GCM encrypted Data Encryption Key string
- `key_version` (INTEGER, default `1`)
- `created_at` (TIMESTAMPTZ, default `now()`)
- `updated_at` (TIMESTAMPTZ, default `now()`, onupdate `now()`)

6.3.4 31.4 `email_verification_tokens` Table

- `id` (UUID, Primary Key)
- `user_id` (UUID, Foreign Key → `users.id` ON DELETE CASCADE, Indexed)
- `token_hash` (VARCHAR(64), Unique, Indexed)
- `otp_code` (VARCHAR(6), Nullable) — 6-digit one-time password
- `attempts` (INTEGER, default `0`)
- `expires_at` (TIMESTAMPTZ)
- `is_used` (BOOLEAN, default `FALSE`)
- `used_at` (TIMESTAMPTZ, Nullable)

6.3.5 31.5 `user_settings` Table

- `user_id` (UUID, Primary Key, Foreign Key → `users.id` ON DELETE CASCADE)
- `folders` (JSONB, default `'[]'`)
- `pinned_conversations` (JSONB, default `'[]'`)
- `conversation_tags` (JSONB, default `'{}'`)
- `document_folders` (JSONB, default `'[]'`)
- `document_folder_mappings` (JSONB, default `'{}'`)
- `document_custom_order` (JSONB, default `'[]'`)
- `document_filter_presets` (JSONB, default `'[]'`)
- `preferences` (JSONB, default `'{}'`)

6.4 32. Chat & Conversation Context (3 Tables & Repositories)

6.4.1 32.1 `conversations` Table

- `id` (UUID, Primary Key)
- `user_id` (UUID, Foreign Key → `users.id` ON DELETE CASCADE, Indexed)
- `title` (VARCHAR(255), default `'New Conversation'`)
- `state` (VARCHAR(50), default `'active'`, Indexed) — `active`, `archived`
- `settings` (JSONB, default `'{}'`, ORM alias `settings_json`)
- `metadata` (JSONB, default `'{}'`, ORM alias `metadata_json`)
- `version` (INTEGER, default 1) — Optimistic Concurrency Control
- `last_message_at` (TIMESTAMPTZ, default `now()`, Indexed)
- `created_at` (TIMESTAMPTZ, default `now()`)
- `updated_at` (TIMESTAMPTZ, default `now()`, Indexed)
- **Indexes**:
  - `ix_conversations_user_last_msg_id` ON `("user_id", last_message_at DESC, id DESC) WHERE state <> 'deleted'` — Partial composite index for *O*(1) active conversation keyset cursor pagination.
  - `ix_conversations_user_id`, `ix_conversations_state`, `ix_conversations_last_message_at`, `ix_conversations_updated_at`.

6.4.2 32.2 `conversation_messages` Table

- `id` (UUID, Primary Key)
- `conversation_id` (UUID, Foreign Key → `conversations.id` ON DELETE CASCADE, Indexed)
- `role` (VARCHAR(20)) — `user`, `assistant`, `system`
- `content` (TEXT)
- `status` (VARCHAR(50), default `'completed'`)
- `attachments` (JSONB, default `'[]'`)
- `citations` (JSONB, default `'[]'`)
- `structured_content` (JSONB, default `'{}'`)
- `tool_calls` (JSONB, default `'[]'`)
- `metadata` (JSONB, default `'{}'`, ORM alias `metadata_json`, captures `execution_duration_ms`, trace IDs, token costs)
- `prompt_tokens` (INTEGER, default `0`)
- `completion_tokens` (INTEGER, default `0`)
- `created_at` (TIMESTAMPTZ, default `now()`, Indexed)
- **Indexes**:
  - `ix_conv_messages_conv_created_id` ON `(conversation_id, created_at DESC, id DESC)` — Composite index for message history keyset cursor pagination.
  - `ix_conversation_messages_conversation_id`, `ix_conversation_messages_created_at`.

6.4.3 32.3 `conversation_summaries` Table

- `id` (UUID, Primary Key)
- `conversation_id` (UUID, Unique, Foreign Key → `conversations.id` ON DELETE CASCADE)
- `summary` (TEXT)
- `summary_version` (INTEGER, default 1)
- `covered_from_message_id` (UUID, Nullable) — Logical boundary without DB-level constraint to prevent cascade locks
- `covered_until_message_id` (UUID, Nullable) — Logical boundary without DB-level constraint
- `message_count` (INTEGER, default `0`)

- `created_at` (TIMESTAMPTZ, default `now()`)

6.5 33. Document Ingestion, Chunks & Storage Context (9 Tables & Repositories)

6.5.1 33.1 `documents` Table

- `id` (UUID, Primary Key)
- `user_id` (UUID, Foreign Key → `users.id` ON DELETE CASCADE, Indexed)
- `conversation_id` (UUID, Foreign Key → `conversations.id` ON DELETE SET NULL, Nullable, Indexed)
- `scope` (VARCHAR(20), default `'library'`) — `library`, `conversation`
- `filename` (VARCHAR(255))
- `original_filename` (VARCHAR(255))
- `file_size` (BIGINT)
- `content_type` (VARCHAR(100))
- `checksum` (VARCHAR(64), Indexed) — SHA-256 file content hash
- `status` (VARCHAR(50), default `'queued'`, Indexed) — `queued`, `processing`, `completed`, `failed`
- `version` (INTEGER, default `1`)
- `active_version` (INTEGER, default `1`)
- `active_processing_config_version_id` (UUID, Nullable)
- `extractor_version` (VARCHAR(50), default `'v1'`)
- `failure_reason` (TEXT, Nullable)
- `word_count` (INTEGER, default `0`)
- `language` (VARCHAR(10), default `'en'`)
- `page_count` (INTEGER, default `0`)
- `processing_started_at` (TIMESTAMPTZ, Nullable)
- `processing_completed_at` (TIMESTAMPTZ, Nullable)
- `created_at` (TIMESTAMPTZ, default `now()`)

6.5.2 33.2 `document_files` Table

- `key` (VARCHAR(1024), Primary Key)
- `content` (BYTEA)

6.5.3 33.3 `document_chunks` Table

- `id` (UUID, Primary Key)
- `document_id` (UUID, Foreign Key → `documents.id` ON DELETE CASCADE, Indexed)
- `chunk_index` (INTEGER)
- `text` (TEXT)
- `token_estimate` (INTEGER, default `0`)
- `character_start` (INTEGER, Nullable)
- `character_end` (INTEGER, Nullable)
- `metadata` (JSON, default `'{}'`, ORM alias `metadata_json`)
- `search_vector` (TSVECTOR, Generated Stored, GIN Indexed)
- `created_at` (TIMESTAMPTZ, default `now()`)

6.5.4 33.4 `document_embeddings` Table

- `id` (UUID, Primary Key)
- `chunk_id` (UUID, Unique, Foreign Key → `document_chunks.id` ON DELETE CASCADE)

- `embedding_provider` (VARCHAR(50), default `'jina'`)
- `embedding_model` (VARCHAR(100), default `'jina-embeddings-v3'`)
- `embedding_dimension` (INTEGER, default 768)
- `embedding_version` (VARCHAR(50), default `'v1'`)
- `status` (VARCHAR(50), default `'completed'`)
- `created_at` (TIMESTAMPTZ, default `now()`)
- `updated_at` (TIMESTAMPTZ, default `now()`, onupdate `now()`) *(Note: Qdrant uses the chunk's UUID directly as the point ID, eliminating point ID sync overhead).*

6.5.5 33.5 `document_elements` Table

- `id` (UUID, Primary Key)
- `document_id` (UUID, Foreign Key → `documents.id` ON DELETE CASCADE, Indexed)
- `user_id` (UUID, Foreign Key → `users.id` ON DELETE CASCADE, Indexed)
- `document_version` (INTEGER, default 1)
- `element_index` (INTEGER)
- `parent_element_id` (UUID, Nullable)
- `element_type` (VARCHAR(50)) — `table`, `figure`, `image`, `header`
- `content` (TEXT)
- `normalized_content` (TEXT, Nullable)
- `bounding_box` (JSONB, default `'{}'`, ORM alias `bounding_box_json`)
- `metadata` (JSONB, default `'{}'`, ORM alias `metadata_json`)
- `page_number` (INTEGER)
- `search_vector` (TSVECTOR, Generated Stored, GIN Indexed)

6.5.6 33.6 `processing_config_versions` Table

- `id` (UUID, Primary Key)
- `config_identifier` (VARCHAR(64), Unique, Indexed)
- `config_json` (JSONB) — Contains `chunk_size`, `chunk_overlap`, `semantic_threshold`
- `created_at` (TIMESTAMPTZ, default `now()`)

6.5.7 33.7 `document_processing_runs` Table

- `id` (UUID, Primary Key)
- `user_id` (UUID, Foreign Key → `users.id` ON DELETE CASCADE, Indexed)
- `document_id` (UUID, Foreign Key → `documents.id` ON DELETE CASCADE, Indexed)
- `document_version` (INTEGER, default 1)
- `processing_config_version_id` (UUID, Foreign Key → `processing_config_versions.id`)
- `state` (VARCHAR(50)) — `PENDING`, `ACTIVE`, `COMPLETED`, `FAILED`
- `validation_gate_passed` (BOOLEAN, default `FALSE`)
- `completed_at` (TIMESTAMPTZ, Nullable)
- `superseded_at` (TIMESTAMPTZ, Nullable)

6.5.8 33.8 `document_processing_jobs` Table

- `id` (UUID, Primary Key)
- `run_id` (UUID, Foreign Key → `document_processing_runs.id` ON DELETE CASCADE, Indexed)
- `user_id` (UUID, Foreign Key → `users.id` ON DELETE CASCADE, Indexed)
- `document_id` (UUID, Foreign Key → `documents.id` ON DELETE CASCADE, Indexed)
- `document_version` (INTEGER, default 1)
- `job_type` (VARCHAR(50)) — `extraction`, `chunking`, `embedding`, `graph_ner`
- `idempotency_key` (VARCHAR(255), Unique, Indexed)

- `lease_token` (VARCHAR(64), Nullable)
- `lease_expires_at` (TIMESTAMPTZ, Nullable)
- `heartbeat_at` (TIMESTAMPTZ, Nullable)
- `status` (VARCHAR(50))
- `attempts` (INTEGER, default `0`)
- `last_error` (TEXT, Nullable)

6.5.9 33.9 `durable_cleanup_jobs` Table

- `id` (UUID, Primary Key)
- `resource_type` (VARCHAR(50)) — `document`, `conversation`, `user`
- `resource_id` (UUID, Indexed)
- `payload` (JSON, default `'{}'`)
- `status` (VARCHAR(50), default `'PENDING'`, Indexed)
- `attempts` (INTEGER, default `0`)
- `last_error` (TEXT, Nullable)
- `created_at` (TIMESTAMPTZ, default `now()`)
- `completed_at` (TIMESTAMPTZ, Nullable)

6.6 34. Knowledge Graph Context (4 Tables & Repositories)

6.6.1 34.1 `graph_entities` Table

- `id` (UUID, Primary Key)
- `name` (VARCHAR(255), Indexed)
- `normalized_name` (VARCHAR(255), Indexed)
- `entity_type` (VARCHAR(100), Indexed) — `PERSON`, `ORGANIZATION`, `METRIC`, `LOCATION`, `CONCEPT`
- `normalized_type` (VARCHAR(100), Indexed)
- `description` (TEXT, Nullable)
- `properties` (JSONB, default `'{}'`, ORM alias `properties_json`)
- `confidence` (FLOAT, default `1.0`)
- `user_id` (UUID, Foreign Key → `users.id` ON DELETE CASCADE, Indexed)
- `created_at` (TIMESTAMPTZ, default `now()`)
- `updated_at` (TIMESTAMPTZ, default `now()`)

6.6.2 34.2 `graph_relationships` Table

- `id` (UUID, Primary Key)
- `source_entity_id` (UUID, Foreign Key → `graph_entities.id` ON DELETE CASCADE, Indexed)
- `target_entity_id` (UUID, Foreign Key → `graph_entities.id` ON DELETE CASCADE, Indexed)
- `relation_type` (VARCHAR(100), Indexed) — `OWNS`, `CALCULATES`, `COMPARES`, `PART_OF`
- `normalized_relation_type` (VARCHAR(100), Indexed)
- `weight` (FLOAT, default `1.0`)
- `user_id` (UUID, Foreign Key → `users.id` ON DELETE CASCADE, Indexed)

6.6.3 34.3 `graph_relationship_evidence` Table

- `id` (UUID, Primary Key)
- `relationship_id` (UUID, Foreign Key → `graph_relationships.id` ON DELETE CASCADE, Indexed)
- `source_element_id` (UUID, Foreign Key → `document_elements.id` ON DELETE CASCADE, Indexed)
- `document_id` (UUID, Foreign Key → `documents.id` ON DELETE CASCADE, Indexed)
- `document_version` (INTEGER, default `1`)

- `status` (VARCHAR(50), default `'active'`)
- `source_span` (TEXT)
- `confidence` (FLOAT, default `1.0`)

6.6.4 34.4 `graph_entity_mentions` Table

- `id` (UUID, Primary Key)
- `entity_id` (UUID, Foreign Key → `graph_entities.id` ON DELETE CASCADE, Indexed)
- `element_id` (UUID, Foreign Key → `document_elements.id` ON DELETE CASCADE, Indexed)
- `document_version` (INTEGER, default `1`)
- `confidence` (FLOAT, default `1.0`)
- `status` (VARCHAR(50), default `'active'`)
- `source_span` (TEXT)

6.7 35. Transactional Outbox Context (1 Table) & Automated Evaluation Context (3 Tables)

6.7.1 35.1 `outbox_events` Table

- `id` (UUID, Primary Key)
- `document_id` (UUID, Nullable, Indexed)
- `document_version` (INTEGER, Nullable)
- `user_id` (UUID, Foreign Key → `users.id` ON DELETE CASCADE, Indexed)
- `event_type` (VARCHAR(100), Indexed) — `document.created`, `document.deleted`, `chat.completed`
- `payload` (JSONB, ORM alias `payload_json`)
- `status` (VARCHAR(50), default `'PENDING'`, Indexed) — `PENDING`, `PUBLISHED`, `FAILED`
- `created_at` (TIMESTAMPTZ, default `now()`, Indexed)
- `published_at` (TIMESTAMPTZ, Nullable)
- *Constraint*: `UNIQUE (user_id, document_id, document_version, event_type)`

6.7.2 35.2 `evaluation_runs` Table

- `id` (UUID, Primary Key)
- `name` (VARCHAR(255))
- `dataset_id` (VARCHAR(100), Indexed)
- `dataset_version` (VARCHAR(50))
- `total_samples` (INTEGER, default `0`)
- `completed_samples` (INTEGER, default `0`)
- `profile` (VARCHAR(100))
- `provider` (VARCHAR(50))
- `model` (VARCHAR(100))
- `temperature` (FLOAT)
- `embedding_model` (VARCHAR(100))
- `reranker` (VARCHAR(100))
- `retrieval_mode` (VARCHAR(50))
- `system_prompt_version` (VARCHAR(50))
- `git_commit` (VARCHAR(64), Nullable)
- `git_branch` (VARCHAR(100), Nullable)
- `total_input_tokens` (INTEGER, default `0`)
- `total_output_tokens` (INTEGER, default `0`)
- `total_cost` (FLOAT, default `0.0`)

- `avg_latency` (FLOAT, default `0.0`)
- `max_latency` (FLOAT, default `0.0`)
- `status` (VARCHAR(50), Indexed)
- `error_message` (TEXT, Nullable)
- `created_at` (TIMESTAMPTZ, default `now()`)
- `completed_at` (TIMESTAMPTZ, Nullable)

6.7.3 35.3 `evaluation_results` Table

- `id` (UUID, Primary Key)
- `run_id` (UUID, Foreign Key → `evaluation_runs.id` ON DELETE CASCADE, Indexed)
- `sample_id` (VARCHAR(100), Indexed)
- `question` (TEXT)
- `expected_answer` (TEXT)
- `actual_answer` (TEXT)
- `latency` (FLOAT, default `0.0`)
- `input_tokens` (INTEGER, default `0`)
- `output_tokens` (INTEGER, default `0`)
- `cost` (FLOAT, default `0.0`)
- `error_category` (VARCHAR(100), Nullable)
- `metrics` (JSONB, default `'{}'`)
- `retrieved_contexts` (JSONB, default `'[]'`)
- `metadata` (JSONB, default `'{}'`, ORM alias `metadata_json`)

6.7.4 35.4 `benchmark_runs` Table

- `id` (UUID, Primary Key)
- `name` (VARCHAR(255))
- `profile` (VARCHAR(100))
- `status` (VARCHAR(50))
- `results_summary` (JSON, default `'{}'`)
- `metadata` (JSON, default `'{}'`, ORM alias `metadata_json`)
- `created_at` (TIMESTAMPTZ, default `now()`)

---

7 Part V: Security, Reliability, Caching & Distributed Infrastructure

7.1 36. Envelope Encryption, Key Management & Crypto-Shredding (AES-256-GCM + DEK/KEK)

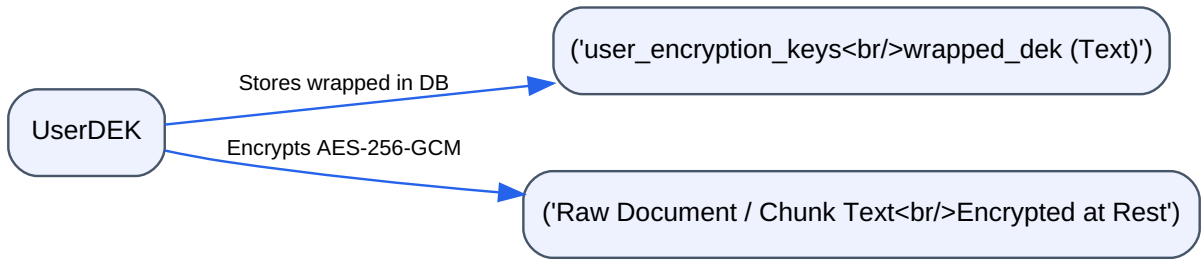


Figure 9 — rendered from the Mermaid diagram in the source document.

When an account is deleted via `DELETE /api/v1/auth/account`, the wrapped DEK in `user_encryption_keys` is permanently purged, achieving instant, provable cryptographic shredding across database backups.

7.2 37. Malware & Antivirus Scanning (ClamAV TCP Streaming + EICAR)

Located in `backend/app/core/security/malware.py`:

- Streamed over a non-blocking TCP socket to `clamd` on port `3310` using `INSTREAM`.
- In-memory heuristic scanner fallback checking for EICAR standard antivirus test signatures and suspicious executable magic bytes (`MZ`, `ELF`).

7.3 38. Prompt Injection Defense, Intent Guardrails & AST Sandboxing

Located in `backend/app/core/security/`:

- **Intent Classifier & Safety Guardrails (`intent_classifier.py`)**:
  - Deterministic and semantic pre-execution intent guardrails (`IntentClassifier`).
  - Classifies user inputs into `STANDARD_QUERY`, `PROMPT_INJECTION`, `SYSTEM_OVERRIDE`, `MALICIOUS_EXECUTION`, or `OUT_OF_SCOPE`.
  - Rejects unsafe queries with HTTP 400 problem details before vector search or LLM consumption.

- **Prompt Injection Defense (`prompt_injection.py`)**:
  - Regular expression and rule-based heuristic scanner checking for delimiter escapes (`</retrieved_context>`, `[SYSTEM]`, `Ignore previous instructions`), role hijacking, and jailbreak patterns.
- **Secret & PII Sanitizer (`secrets.py`, `logger.py`)**:
  - Comprehensive regex masking intercepting log streams and trace attributes:
    - Redacts `Bearer <jwt>`, `gemini_api_key`, `jina_api_key`, `postgres://...` credentials, and raw email addresses.
- **Sliding-Window Rate Limiter (`rate_limiter.py`)**:
  - Multi-tier sliding-window algorithm backed by Redis with local in-memory fallback.
  - Supports IP-based rate limiting (60 req/min), user account tier limiting, and sensitive endpoint throttling (account deletion, login).
- **Distributed Request Idempotency (`idempotency.py`)**:
  - Uses atomic Redis `SET ... NX EX` locks (`IdempotencyService`) to prevent concurrent duplicate file ingestion jobs and double token streaming requests.
- **Brute-Force Login Lockout Engine (`auth.py`)**: Tracks attempts keyed by `ip:{ip}:email:{email}` in Redis; locks account for 300s after 5 consecutive failures.
- **ASGI Security Headers**:
  - `Content-Security-Policy: default-src 'self'; frame-ancestors 'none'; object-src 'none';`
  - `X-Frame-Options: DENY`
  - `X-Content-Type-Options: nosniff`
  - `Strict-Transport-Security: max-age=31536000; includeSubDomains`
- **AST SafeCalculator (`sandboxing.py`, `backend/app/core/math/sandbox.py`)**: Mathematical evaluation parser rejecting all dunder attributes and arbitrary calls.
- **Global Exception Handlers (`exception_handlers.py`)**: Maps domain exceptions (`AuthenticationError`, `QuotaExceededError`, `MalwareDetectedError`, `SecurityViolationError`) to RFC-7807 problem details.

7.4 39. Multi-Tier Caching (L1 Memory + L2 Redis) & Distributed Heartbeat Locking

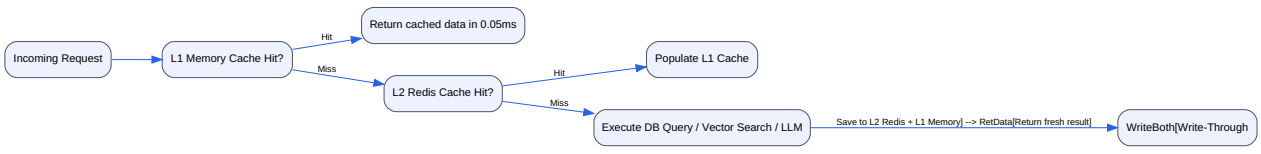


Figure 10 — rendered from the Mermaid diagram in the source document.

- **L1 In-Memory Cache**: LRU eviction (N = 1000 entries, TTL=60s).
- **L2 Redis Cache**: Partitioned by 6 managed namespaces (`prompt_rewrite`, `embeddings`, `conversation_memory`, `retrieval_results`, `evaluation_benchmarks`, `tool_executions`).

- **Distributed Heartbeat Lock (`lock.py`)**: Mutex with background coroutine extending TTL during active long-running jobs.

7.5 40. Reliability Framework: Bulkheads, Circuit Breakers, Exponential Jitter Retries & Timeouts

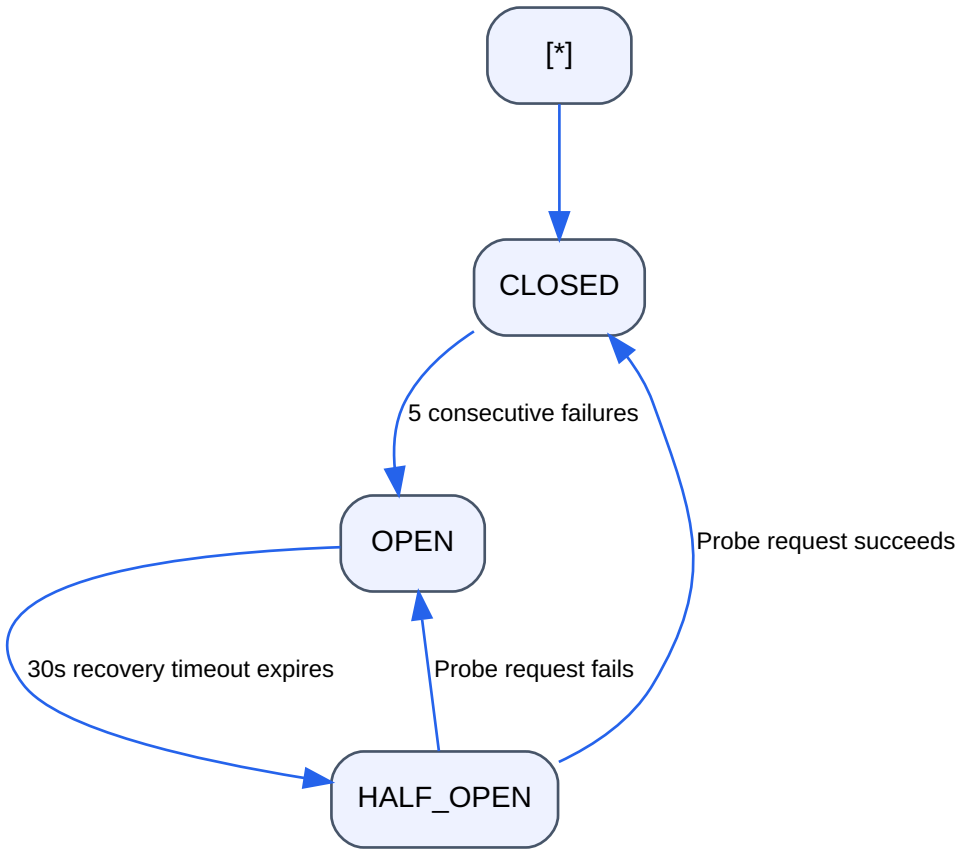


Figure 11 — rendered from the Mermaid diagram in the source document.

7.6 41. Transactional Outbox Pattern, Event Bus & JobDispatcher Engine

Located in `backend/app/core/events/` and `backend/app/core/queue/`:

- **Transactional Outbox (`outbox_dispatcher.py`)**: Guaranteed message delivery through atomic PostgreSQL commits with idempotency constraints.
- **Redis Event Bus (`RedisEventBus`)**: PubSub event broadcasting for domain subscribers (`document.created`, `user.registered`).
- **JobDispatcher & Handlers (`dispatcher.py`, `handlers.py`)**:
  - `ProcessDocumentJobHandler`: Manages extraction, chunking, and NER linking across 6 stages.
  - `GenerateEmbeddingsJobHandler`: Dispatches batch embeddings to Jina and upserts to Qdrant.

- `SummarizeConversationJobHandler`: Executes asynchronous rolling summarization.

7.7 42. Storage Provider Abstraction & Two-Phase Durable Cleanup Worker

Located in `backend/app/documents/services/cleanup_worker.py` and `app/documents/infrastructure/storage.py`:

- **Storage Providers**: Supported via `StorageProvider` abstraction with native production engines: `database` (BYTEA storage in `document_files` / Supabase), and `local` (`data/uploads`).
- Two-Phase Durable Cleanup executes a 2PC cascade delete across underlying object storage/blobs, Qdrant vector payload points, and PostgreSQL metadata rows.

7.8 43. Transactional Email, Request Idempotency, Pricing Engine & Resource Quotas

- **Transactional Email (`resend_service.py`)**: Integration with Resend REST API with styled HTML templates (`_build_magic_link_html`, `_build_verification_email_html`) and 6-digit OTP delivery.
- **Distributed Idempotency (`idempotency.py`)**: Redis-backed fingerprinting (`IdempotencyService`) preventing duplicate uploads and redundant LLM executions.
- **Real-Time Pricing Engine (`pricing_provider.py`, `pricing.yaml`)**: `YAMLPricingProvider` parses model token rates (e.g. `gemini-3.5-flash-lite`: $0.10 input / $0.40 output per 1M tokens) and calculates USD consumption on every stream completion.
- **Request Context Propagation (`request_context.py`)**: `RequestContext` dataclass (`user_id`, `role`, `request_id`, `client_ip`) propagated via `contextvars.ContextVar` across async tasks.
- **Structured Logging (`logger.py`)**: Contextual log filters with JSON/TEXT switch and automatic masking of API keys, Authorization headers, and emails.
- **Resource Quotas (`quotas.py`)**: Sliding-window tracking of `USER_DAILY_TOKEN_LIMIT` (1M tokens) and `USER_MAX_STORAGE_BYTES` (500MB).
- **Dynamic Feature Flags (`flags.py`)**: Runtime feature flag engine evaluating environment overrides (`ENABLE_PARENT_CHILD_RAG`, `ENABLE_WEIGHTED_RRF`, `ENABLE_INTENT_GUARDRAILS`).

7.9 44. Application Lifespan, Startup Hooks & Middleware Cardinality Defense

Located in `backend/app/main.py` and `backend/app/core/startup.py`:

- **Lifespan Startup Tasks (`startup.py`)**:
  - `seed_admin_user`: Auto-provisions system administrator on initial startup.
  - `cleanup_stale_unverified_signups`: Purges abandoned unverified accounts older than 7 days.
  - `run_durable_cleanup_recovery`: Reclaims orphaned two-phase deletion jobs.
  - `verify_alembic_migrations`: Verifies database revision matches codebase.
  - Pre-warms Jina cross-encoder network sessions.
- **Middleware Cardinality Defense (`resolve_route_template()`)**: Normalizes raw UUID path variables into `/api/v1/conversations/{id}` to prevent Prometheus metric cardinality explosions.

---

8 Part VI: Document Ingestion, Advanced Chunking & Retrieval Intelligence

8.1 45. Ingestion Worker Lease Protocol, Multi-Phase Pipeline & Durable Reaper

8.1.1 45.1 Multi-Phase Ingestion State Machine (`ProcessDocumentJobHandler`)

1. **Stage 1 (File Retrieval)**: Retrieves raw binary payload from configured `StorageProvider` (`database`, `local`).
2. **Stage 2 (Parsing)**: Invokes `ExtractorFactory` (`PDFExtractor`, `DocxExtractor`, `ImageVisionExtractor`, `MarkdownExtractor`, `PlainTextExtractor`).
3. **Stage 3 (Knowledge Graph NER)**: Extracts entity triplets into `graph_entities` and `graph_relationships`.
4. **Stage 4 (Chunking)**: Generates character-bounded chunks with token estimations and full-text GIN search vectors.
5. **Stage 5 (Embedding Dispatch)**: Enqueues `GENERATE_EMBEDDINGS` task for Qdrant indexing.
6. **Stage 6 (Cache Invalidation)**: Clears empty knowledge base cache for `user_id`.

8.1.2 45.2 Ingestion Lease, Heartbeat & Durable Reaper

- Ingestion jobs in `document_processing_jobs` use lease tokens (`lease_token`), expiration timestamps (`lease_expires_at`), and active worker heartbeats (`heartbeat_at`).
- `DurableIngestionReaper` runs every 60s (`INGESTION_REAPER_INTERVAL_SECONDS=60.0`), reclaiming expired worker leases and routing failed jobs to `stream:jobs:dead_letter` after 3 attempts (`QUEUE_MAX_ATTEMPTS=3`).

8.2 46. Advanced Chunking Strategies: Recursive, Parent-Child & Sentence-Window

Located in `backend/app/documents/services/chunking.py`:

1. **Recursive Character Chunking (Default)**: `DEFAULT_CHUNK_SIZE=500`, `DEFAULT_CHUNK_OVERLAP=50`.
2. **Parent-Child Chunking**: `PARENT_CHUNK_SIZE=1500`, `CHILD_CHUNK_SIZE=400`.
3. **Sentence-Window Chunking**: `SENTENCE_WINDOW_SIZE=2`.

8.3 47. Empty Knowledge Base Index Cache & Weighted RRF Fusion

- **Empty Knowledge Base Index Cache (`index_cache.py`)**: Caches whether a `user_id` has indexed chunks (`EMPTY_KB_CACHE_TTL_SECONDS=60.0`), bypassing vector/sparse queries for users with 0 documents.
- **Weighted Reciprocal Rank Fusion (`fusion.py`)**: $$ \text{Score}_{\text{weighted}}(d) = w_{\text{dense}} \cdot \frac{1}{k + r_{\text{dense}}(d)} + w_{\text{sparse}} \cdot \frac{1}{k + r_{\text{sparse}}(d)} $$

8.4 48. Document Parsers, Multimodal OCR, Tables & Metadata Extractors

Located in `backend/app/documents/services/`:

- **Magic Byte Signatures**:
  - `PDF`: `%PDF-` (`0x25 0x50 0x44 0x46 0x2D`)
  - `DOCX`: `PK\x03\x04` (ZIP Container Header)
  - `PNG`: `\x89PNG\r\n\x1a\n`
  - `JPEG`: `\xFF\xD8\xFF`
- **Extractors Hierarchy (`extractors.py`)**:
  - `PDFExtractor`: Digital extraction via `pypdf` with automated OCR failover.
  - `DocxExtractor`: Structured heading and table extraction.
  - `ImageVisionExtractor`: Multimodal OCR via Gemini Vision converting screenshots into markdown tables/text.
- **Specialized Processors**:
  - `OCRService` (`ocr_service.py`): Gemini Vision multimodal OCR with bounding box coordinates.
  - `TableProcessor` (`table_processor.py`): Markdown/CSV table parsing and statistical column metrics.
  - `VisionProcessor` (`vision_processor.py`): Image classification into `FIGURE`, `CHART`, `DIAGRAM`, `TABLE_SNAPSHOT`.
- **Zero-Downtime Replacement & Promotion Gate (`PUT /documents/{id}`)**: Increments document `version`, completes background chunking and vector indexing, and atomically swaps `active_version` on validation pass (`validation_gate_passed = True`).

8.5 49. Query Intelligence: Classification, Decomposition, Expansion, HyDE & ContextSelector

Located in `backend/app/documents/services/`:

- **`DeterministicQueryClassifier` (`query_classifier.py`)**: Regex-based token intent classifier.
- **`DeterministicQueryDecomposer` (`query_decomposer.py`)**: Compound sub-question splitter.
- **`QueryExpansionService` (`query_expansion.py`)**: Acronym and synonym enrichment.
- **`SafeHyDEExecutor` (`hyde.py`)**: Synthetic hypothetical passage generation.
- **`MMR` (`mmr.py`)**: Maximal Marginal Relevance diversity filtering ($\lambda = 0.5$).
- **`ContextSelector` (`context_selector.py`)**:
  - Strict token budget manager (`CONTEXT_MAX_TOKENS=6000`, `MAX_RETRIEVAL_CHUNKS=12`).
  - Resolves Parent-Child relationships to inject parent element contexts for child chunk hits.
  - Deduplicates retrieved passages across hybrid search branches before prompt construction.

8.6 50. Ingestion-Time NER, Relationship Extraction & BFS Knowledge Graph Retrieval

- LLM entity/triplet extraction persisting to `graph_entities` and `graph_relationships`.
- Recursive CTE BFS traversal in PostgreSQL exploring multi-hop entity connections up to depth 3.

8.7 51. Adaptive RAG Orchestration State Machine (DIRECT, HYDE, MULTI_HOP, SPECIALIST)

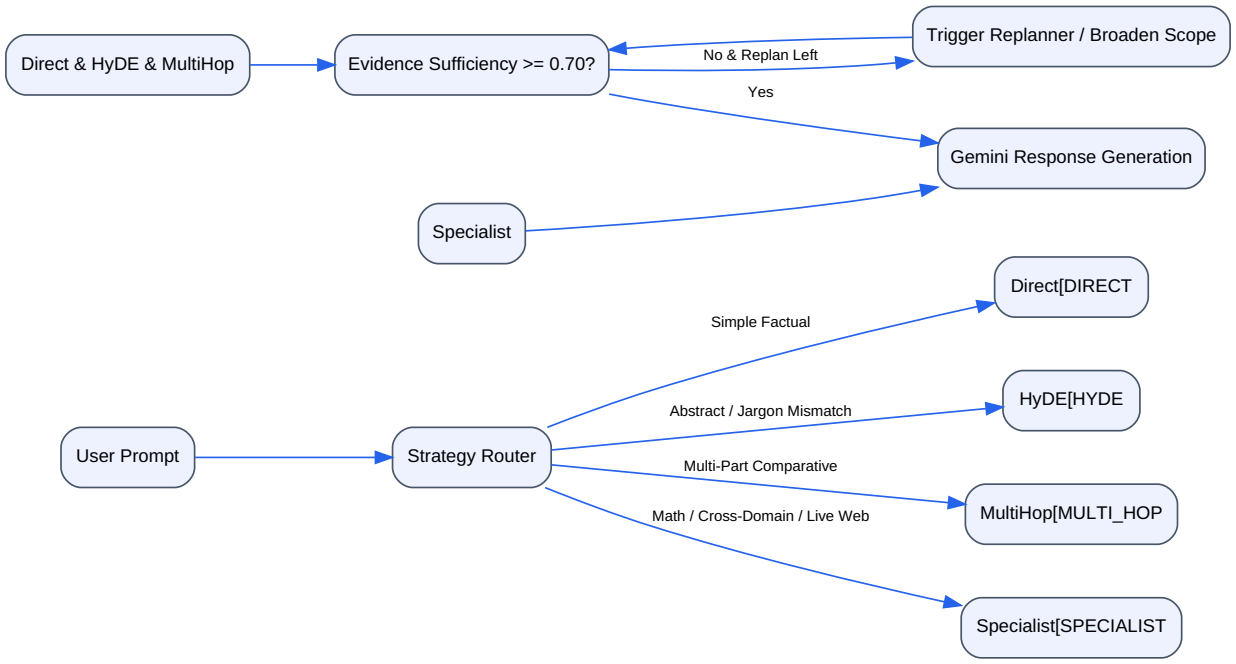


Figure 12 — rendered from the Mermaid diagram in the source document.

9 Part VII: Frontend Architecture, Pyodide WASM & UI Subsystems

9.1 52. Pyodide WebAssembly (WASM) Sandbox Worker Protocol & Managers

Located in `frontend/src/workers/pyodide.worker.ts` and `frontend/src/lib/pyodide/pyodideManager.ts`:

- **Web Worker Message Schema**:
  - Actions: `INIT`, `RUN_CODE`, `INTERRUPT`.
  - Responses: `INIT_SUCCESS`, `STDOUT`, `STDERR`, `PLOT_PNG` (base64 PNG data URLs from Matplotlib `BytesIO`), `RESULT`, `ERROR`.
- **Preloaded Environment**: In-browser virtual VM loaded with `numpy`, `pandas`, `matplotlib`, and `sympy`.

9.2 53. Complete Custom Hook Architecture & State Context Providers

9.2.1 53.1 Frontend Context Providers

- **`UploadQueueContext.tsx`**: Multi-file progressive upload coordinator managing concurrency, per-file stage progress (1-7), error recovery, and cancellation tokens.
- **`AuthContext.tsx`**: In-memory JWT storage, 14-minute silent token refresh via HTTP-only cookie, and global login modal triggers.
- **`ThemeContext.tsx`**: Dark/Light mode tokens and system preference syncing.

9.2.2 53.2 Complete Custom Hook Inventory

- `useAuth.ts`: Authentication actions, login/logout, and user profile state.
- `useConversations.ts`: Thread creation, keyset cursor pagination (`next_cursor`, `prev_cursor`, `has_more`), pinning, and deletion.
- `useDocuments.ts`: Document library querying, filtering, and deletion.
- `useSSEChat.ts`: Streaming SSE consumption and citation tagging.
- `useDrafts.ts`: Message draft persistence across conversation switching via browser `localStorage`.
- `useLocalConversationMetadata.tsx`: Optimistic titles, unread badges, and last message timestamps before server sync.
- `useKeyboardShortcuts.ts`: Global keyboard hotkeys (`Cmd/Ctrl+K`, `Cmd/Ctrl+/`).
- `useFileValidation.ts`: Client-side magic bytes, MIME, and size validation before network dispatch.
- `useOnlineStatus.ts`: Browser navigator online/offline listener.
- `useMessages.ts`: Message state with bidirectional cursor pagination (`next_cursor`, `prev_cursor`), streaming tokens assembly, and branch deletion rollback handlers.
- `useEmbeddingStatus.ts`: Polling vector indexing completion status.
- `useRecentAttachments.ts`: Recent upload attachments management for fast query reuse.
- `useSidebarState.ts`: Collapsible sidebar layout state and mobile drawer sync.
- `useUserSettings.ts`: User preferences, folders, filter presets, and theme sync.
- `useDocumentPipelineSummary.ts` & `useDocumentStatusMap.ts`: Real-time pipeline status aggregation.

9.3 54. 7-Stage Progressive Upload Pipeline & State Machine

Located in `frontend/src/lib/uploadStages.ts` and `frontend/src/components/layout/SidebarUploadIndicator.tsx`:

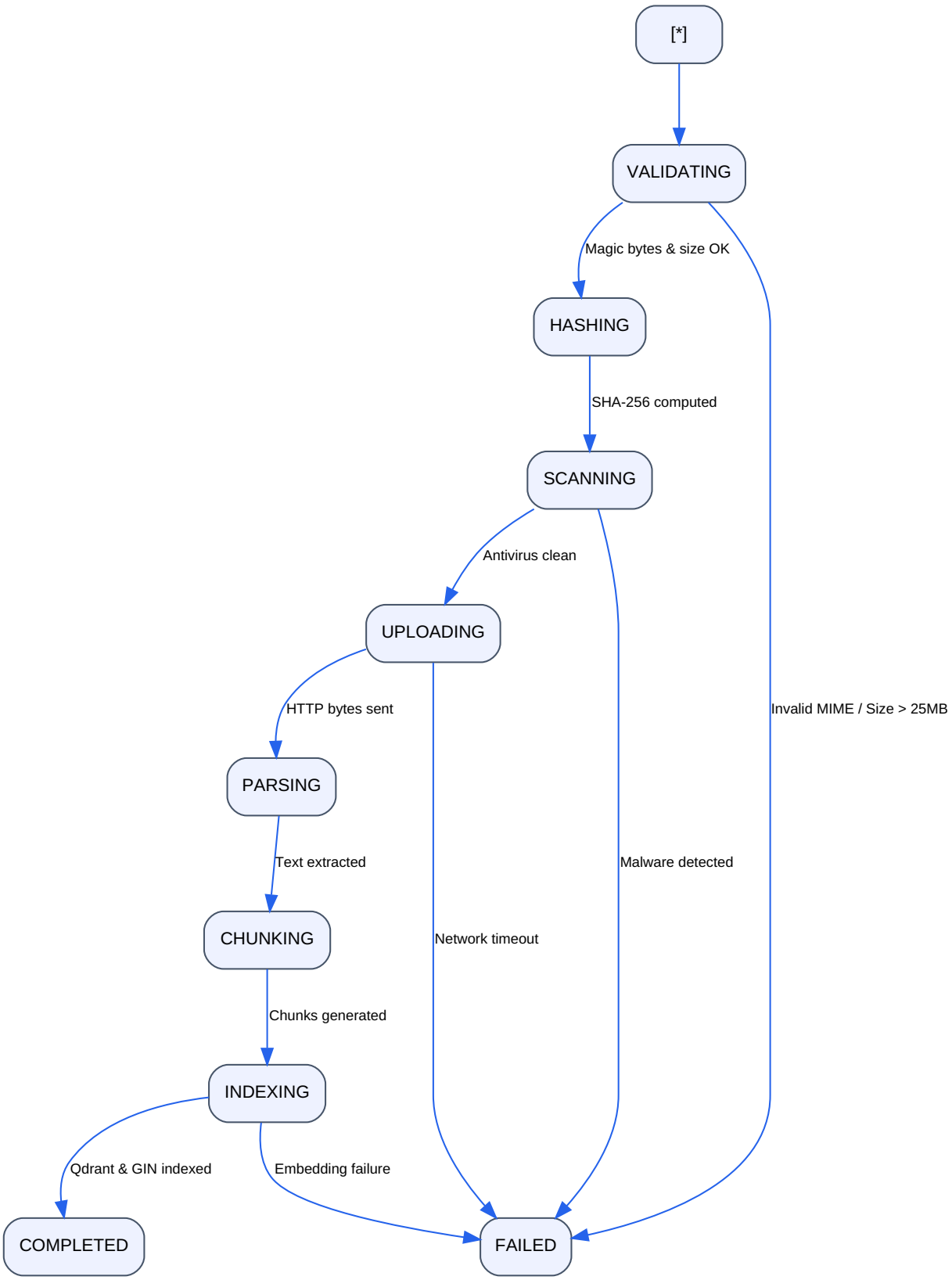


Figure 13 — rendered from the Mermaid diagram in the source document.

9.4 55. Application Pages, Route Guards, Projects & Settings Subsystems

- **Route Guards (`frontend/src/routes/`)**: `AdminRoute.tsx` (admin role check), `ProtectedRoute.tsx` (auth check & unverified redirect), `PublicRoute.tsx`.
- **Application Pages**:
  - `ChatPage.tsx`: Full-screen chat with document chips and Pyodide widgets.
  - `DocumentsPage.tsx`: Document management with bulk operations.
  - `SettingsPage.tsx`: Preferences, appearance, API keys, password, and encryption keys.
  - `ProjectsPage.tsx`: Workspace isolation and folder hierarchy.
  - `AdminDashboardPage.tsx`: Queue lag monitors, cache flusher, and eval runner.
  - `LandingPage.tsx`: Public marketing landing page.
  - `VerifyEmailPage.tsx` & `OAuthCallbackPage.tsx`: Token exchange and OTP verification.

9.5 56. UI Component Tree, Axios Interceptors, Stream Recovery & CodeInterpreter

- **Axios HTTP Client (`frontend/src/api/client.ts`)**: Global interceptors attaching `X-Request-ID`, bearer tokens, 401 token refresh queue with in-flight buffering, and RFC-7807 problem details parsing.
- **Code Interpreter Output (`CodeInterpreterOutput.tsx`)**: Displays WASM status, execution duration, terminal stdout/stderr, and renders interactive Matplotlib base64 PNG charts.
- **Stream Interruption Recovery (`StreamingRecovery.tsx`)**: Buffers tokens during network drops and provides an inline resume action.
- **Re-Authentication Modal (`ReauthDialog.tsx`)**: Re-authenticates expired sessions without discarding unsent drafts.
- **Onboarding Tour (`OnboardingTour.tsx`)**: Interactive onboarding guide for first-time users.
- **Command Palette (`CommandPalette.tsx`)**: Global `Cmd+K` searchable modal.
- **Core UI Components**:
  - **Chat Subsystem**: `ChatInput`, `ChatMessage`, `ChatToolbar`, `ChatWindow`, `CitationCard`, `CitationDrawer`, `CodeBlock` (Shiki syntax highlighting), `CodeInterpreterOutput`, `ConversationDocumentChips`, `EmptyChat`, `MarkdownRenderer`, `MessageList`, `StreamingCursor`, `StreamingRecovery`, `ToolCallCard`.
  - **Document Subsystem**: `DocumentCard`, `DocumentFilters`, `DocumentGrid`, `UploadDropzone`.
  - **Layout & Sidebar**: `ChatLayout`, `MobileDrawer`, `OfflineBanner`, `Sidebar`, `SidebarConversationList`, `SidebarFolders`, `SidebarHeader`, `SidebarProfile`, `SidebarUploadIndicator`.
  - **Auth Forms**: `LoginCard`, `LoginForm`, `ReauthDialog`, `RegisterCard`, `RegisterForm`.
  - **Shadcn/UI Primitives**: `Avatar`, `Button`, `Dropdown`, `FileTypeBadge`, `Input`, `Modal`, `SearchBox`, `UploadProgress`, `badge`, `card`, `command`, `scroll-area`, `separator`, `sheet`, `skeleton`, `states`, `textarea`, `tooltip`.
  - **SEO & Error**: `ErrorBoundary`, `ErrorPage`, `OfflinePage`, `SEOHead`.
- **Dynamic Chunk Retry (`lazyWithRetry.ts`)**: Automatically retries failed dynamic imports on new deployments.

---

10 Part VIII: Complete REST & SSE API Endpoint Catalog (Exact Codebase Routes & DTOs)

10.1 57. Exhaustive Route Specifications Across All 10 Routers & Root App

10.1.1 57.1 Core Pydantic DTO Schema Definitions

- **`CursorPage[T]`**: `{ items: list[T], limit: int, next_cursor: str | None, prev_cursor: str | None, has_next_page: bool, has_prev_page: bool, has_more: bool }`
- **`VerifyEmailSchema`**: `{ email: str, token: str | None, otp: str | None }`
- **`ResendVerificationSchema`**: `{ email: str }`
- **`MagicLinkRequestSchema`**: `{ email: str }`
- **`UpdatePasswordSchema`**: `{ current_password: str | None, new_password: str }`
- **`DeleteAccountSchema`**: `{ password: str }`
- **`ChatRequest`**: `{ question: str, conversation_id: UUID | None, stream: bool = True, document_ids: list[UUID] | None = None }`
- **`SearchRequest`**: `{ query: str, conversation_id: UUID | None = None, top_k: int = 20, document_ids: list[UUID] | None = None, filters: dict | None = None, retrieval_mode: str = "hybrid" }`
- **`UpdateUserSettingsSchema`**: Partial JSONB updates for `folders`, `pinned_conversations`, `document_folders`, `preferences`.
- **`EmbeddingStatusResponse`**: `{ status: str, chunks_total: int, embedded_chunks: int, failed_chunks: int, provider: str, model: str }`
- **`RunBenchmarkRequest`**: `{ dataset_path: str, dataset_id: str, version: str, profile: str = "quick", pipelines: list[str] | None = None, mock: bool = False }`
- **`RegressionRequest`**: `{ candidate_run_id: str, baseline_run_id: str, thresholds: dict[str, float] | None = None }`
- **`DeepHealthResponse`**: `{ status: str, version: str, database: dict, redis: dict, vector_db: dict, storage: dict, llm: dict }`

```
================================================================================================================
                                    COMPLETE REST & SSE API ENDPOINT CATALOG
================================================================================================================
HTTP Method  Route Endpoint                                  Auth Required  Description & Payload
----------------------------------------------------------------------------------------------------------------
1. Root App (app/main.py)
GET          /                                               No             API discovery root and version metadata
GET          /health                                         No             Basic uptime liveness check
GET          /metrics                                        No             Prometheus operational metrics scrape endpoint
GET          /robots.txt                                     No             Search crawler exclusion headers

2. Authentication Router (app/api/v1/auth.py)
POST         /api/v1/auth/register                           No             Register new user (email, password) -> sends OTP
POST         /api/v1/auth/login                              No             Password login -> JWT + HttpOnly refresh cookie
POST         /api/v1/auth/magic-link                         No             Passwordless magic sign-in link generation
POST         /api/v1/auth/refresh                            Cookie         Rotate refresh token and issue new 15-min JWT
POST         /api/v1/auth/logout                             Yes            Revoke refresh session and clear cookies
POST         /api/v1/auth/password                           Yes            Authenticated password update (UpdatePasswordSchema)
POST         /api/v1/auth/verify-email                       No             Verify signup token hash or 6-digit OTP code
POST         /api/v1/auth/resend-verification                No             Resend email verification link
GET          /api/v1/auth/oauth/google                       No             Initiate Google OAuth2 flow (sets state cookie)
GET          /api/v1/auth/oauth/google/callback              No             Google OAuth2 token exchange (#token= redirect)
GET          /api/v1/auth/oauth/github                       No             Initiate GitHub OAuth2 flow (sets state cookie)
GET          /api/v1/auth/oauth/github/callback              No             GitHub OAuth2 token exchange (#token= redirect)
GET          /api/v1/auth/me                                 Yes            Get current user profile and role
DELETE       /api/v1/auth/account                            Yes            Account deletion with password verification

3. Chat & Streaming Router (app/api/v1/chat.py)
POST         /api/v1/chat                                    Yes            Server-Sent Events (SSE) streaming chat endpoint

4. Search Router (app/api/v1/search.py)
POST         /api/v1/search                                  Yes            Unified hybrid retrieval (Dense + Sparse + RRF + MMR)
```

```
5. Conversations Router (app/api/v1/conversations.py)
GET          /api/v1/conversations                                 Yes           List user conversations with keyset pagination (?cursor=, ?
limit=20) -> CursorPage
GET          /api/v1/conversations/{id}                            Yes           Get conversation details and active document filters
PATCH        /api/v1/conversations/{id}                            Yes           Update title, pinned status, or folder metadata
DELETE       /api/v1/conversations/{id}                            Yes           Delete conversation and schedule cleanup
GET          /api/v1/conversations/{id}/messages                   Yes           Paginated messages with keyset pagination (?cursor=, ?limit=50) ->
CursorPage
DELETE       /api/v1/conversations/{id}/messages/{message_id}      Yes           Message rollback/branching deletion
POST         /api/v1/conversations/{conversation_id}/documents     Yes           Upload document scoped directly to conversation
GET          /api/v1/conversations/{conversation_id}/documents     Yes           List documents scoped to conversation

6. Documents Router (app/api/v1/documents.py)
POST         /api/v1/documents/upload                              Yes           Multipart file upload (PDF/DOCX/TXT/MD/PNG)
GET          /api/v1/documents                                     Yes           List documents with filters (skip, limit, scope)
GET          /api/v1/documents/{id}                                Yes           Get document metadata, status, and chunk statistics
PUT          /api/v1/documents/{id}                                Yes           Replace document with automated version bumping
DELETE       /api/v1/documents/{id}                                Yes           Delete document and queue cascade cleanup
POST         /api/v1/documents/{id}/reprocess                      Yes           Trigger re-chunking and re-indexing
POST         /api/v1/documents/{id}/embeddings/regenerate          Yes           Manual trigger for vector embedding regeneration
DELETE       /api/v1/documents/{id}/embeddings                     Yes           Soft/hard purge of chunk embeddings
GET          /api/v1/documents/{id}/embeddings/status              Yes           Real-time tracking of vector indexing state

7. Evaluation Router (app/api/v1/evaluation.py)
POST         /api/v1/evaluation/benchmark                          Admin         Trigger synthetic benchmark suite run
POST         /api/v1/evaluation/regression                         Admin         Execute regression verification on golden dataset
GET          /api/v1/evaluation/runs                               Admin         List evaluation run histories
GET          /api/v1/evaluation/reports/{run_id}                   Admin         Get evaluation report (format: markdown/json/csv/html)

8. Observability Router (app/api/v1/observability.py)
GET          /api/v1/observability/metrics                         Admin         Retrieve aggregated multi-agent & quality metrics
GET          /api/v1/observability/traces                          Admin         List recent assistant message execution traces (?limit=50, ?
offset=0)

9. Health & Diagnostics Router (app/api/v1/health.py)
GET          /api/v1/health/liveness                               No            Kubernetes liveness probe (Process alive)
GET          /api/v1/health/readiness                              No            Readiness probe (Checks DB, Redis, Qdrant, Jina)
GET          /api/v1/health/deep                                   No            Deep health check across all downstream engines

10. User Settings Router (app/api/v1/user_settings.py)
GET          /api/v1/user/settings                                 Yes           Get user JSONB settings, folders, and preferences
PATCH        /api/v1/user/settings                                 Yes           Update theme, folders, presets, and preferences

11. Admin Router (app/api/v1/admin.py)
POST         /api/v1/admin/cache/clear                             Admin         Dynamic cache clearing (?namespace=) across 6 pools
GET          /api/v1/admin/operational-status                      Admin         Real-time system diagnostics and worker status
================================================================================================================================
```

11 Part IX: Production Observability, Quality Assurance & Evaluation Suite

11.1 58. OpenTelemetry Tracing, Prometheus Metric Registry & Telemetry Sanitizer

11.1.1 58.1 Complete Prometheus Metric Registry

```
================================================================================================================================
                                        PROMETHEUS METRIC REGISTRY
================================================================================================================================
Metric Name                       Type          Labels / Dimensions           Description
--------------------------------------------------------------------------------------------------------------------------------
http_requests_total               Counter       method, route, status_code    Total HTTP request volume by endpoint
http_request_duration_seconds     Histogram     method, route, status_code    Latency distribution across ASGI routes
http_active_requests              Gauge         None                          In-flight concurrent requests
rag_retrieval_duration_seconds    Histogram     strategy, provider            Retrieval branch latency (dense/sparse/rerank)
rag_reranker_scores               Histogram     model                         Distribution of cross-encoder confidence scores
agent_dag_execution_seconds       Histogram     plan_id, status               Total multi-agent DAG execution time
agent_task_execution_seconds      Histogram     agent_type, status            Individual specialist task latency
worker_queue_lag                  Gauge         queue_name                    Redis Stream unacknowledged message depth
llm_token_usage_total             Counter       model, token_type             Prompt and completion tokens consumed
llm_estimated_cost_usd_total      Counter       model                         Accumulated LLM operational cost in USD
================================================================================================================================
```

11.2 59. Multi-Agent Observability & Quality Metrics (AgentObservabilityService)

Tracks DAG execution depth, concurrency factors, task success rates, evidence sufficiency scores, and duplicate queries avoided.

11.3 60. Comprehensive Evaluation Framework: 6 Evaluators, 5 Runners & 33 Metrics

11.3.1 60.1 Dedicated Evaluator Services (`app/evaluation/services/`)

1. **`AgentEvaluator`**: Specialist agent task completion and planning efficiency.
2. **`CitationEvaluator`**: Citation validity, precision, grounding, and coverage.
3. **`ConversationEvaluator`**: Multi-turn context retention, pronoun rewriting, and memory summarization.
4. **`MultiAgentEvaluator`**: DAG topological validity, wave concurrency factors, and replan convergence.
5. **`RetrievalEvaluator`**: Hit Rate, MRR, NDCG@K, Context Recall, and Context Precision.
6. **`ToolEvaluator`**: Single-agent tool calling accuracy, argument validation, and necessity.

11.3.2 60.2 Evaluation Runners (`app/evaluation/runners/`)

- **`EvaluationRunner` (`evaluation_runner.py`)**: Core batch orchestrator running evaluation loops, coordinating providers, and aggregating metrics across datasets.
- **`RagasProvider` (`ragas_provider.py`)**: Ragas integration for Faithfulness and Answer Relevancy.
- **`DeepEvalProvider` (`deepeval_provider.py`)**: DeepEval and G-Eval frameworks.
- **`BenchmarkRunner` (`benchmark_runner.py`)**: Batch execution across the 4 profiles (`quick`, `standard`, `full`, `nightly`).

- **`RegressionRunner` (`regression_runner.py`)**: Dual-Gate PR regression enforcer (Faithfulness ≥ 0.90, $\Delta \leq$ 0.03).

11.3.3 60.3 Complete Catalog of All 33 Registered Metrics in `backend/app/evaluation/metrics/registry.py`

```
================================================================================================================
                                    ALL 33 REGISTERED EVALUATION METRICS
================================================================================================================
Metric Name                    Registry Decorator Key             Category & Technical Formula / Description
----------------------------------------------------------------------------------------------------------------
1. Hit Rate                    @MetricRegistry.register("hit_rate")                    I(any relevant chunk in Top-K)
2. Precision@K                 @MetricRegistry.register("precision_at_k")              Relevant Chunks in Top-K / K
3. kRecall@K                    @MetricRegistry.register("recall_at_k")                 Found Relevant / Expected Relevant
4. Mean Reciprocal Rank        @MetricRegistry.register("mrr")                         1 / Rank of first relevant chunk
5. Normalized DCG              @MetricRegistry.register("ndcg")                        DCG@K / IDCG@K discounted rank score
6. Citation Validity           @MetricRegistry.register("citation_validity")           Valid UUID Citations / Emitted Citations
7. Citation Precision          @MetricRegistry.register("citation_precision")          Ground Truth Citations / Total Citations
8. Citation Grounding          @MetricRegistry.register("citation_grounding")          NLI semantic entailment (0.0 - 1.0)
9. Citation Coverage           @MetricRegistry.register("citation_coverage")           Supported Claims / Total Factual Claims
10. Task Success Rate          @MetricRegistry.register("task_success_rate")           Completed Tasks / Planned Tasks
11. Evidence Sufficiency Rate  @MetricRegistry.register("evidence_sufficiency_rate")   Mean cross-attention score >= 0.70
12. Replan Success Rate        @MetricRegistry.register("replan_success_rate")         I(Replan iteration achieved accept)
13. Duplicate Task Avoidance   @MetricRegistry.register("duplicate_task_avoidance")    Pruned Redundant Queries / Decomposed Queries
14. Agent Call Efficiency      @MetricRegistry.register("agent_call_efficiency")       1.0 - (Redundant Calls / Total Calls)
15. Reasoning Steps            @MetricRegistry.register("reasoning_steps")             Ratio of DAG depth to task complexity
16. Tool Necessity             @MetricRegistry.register("tool_necessity")              Binary verification that tool was required
17. Tool Ordering              @MetricRegistry.register("tool_ordering")               Verification of prerequisite tool execution
18. Reasoning Efficiency       @MetricRegistry.register("reasoning_efficiency")        Reasoning step density per unit complexity
19. Tool Success Rate          @MetricRegistry.register("tool_success_rate")           Successful Tool Executions / Attempted Calls
20. Tool Choice Correctness    @MetricRegistry.register("tool_choice_correctness")     Semantic match of tool to task requirement
21. History Rewriting          @MetricRegistry.register("history_rewriting")           Entailment proving pronoun resolution
22. Context Budget Utilization @MetricRegistry.register("context_budget_utilization")  Tokens consumed vs. 6000 token limit
23. Context Recall             @MetricRegistry.register("context_recall")              Ground truth facts recovered in candidate chunks
24. Context Precision          @MetricRegistry.register("context_precision")           Precision of context windows containing facts
25. Duplicate Context Rate     @MetricRegistry.register("duplicate_context_rate")      Redundant context chunks passing through MMR
26. Strategy Selection Accuracy @MetricRegistry.register("strategy_selection_accuracy") Optimal strategy chosen (DIRECT/HYDE/etc.)
27. HyDE Usage Rate            @MetricRegistry.register("hyde_usage_rate")             Frequency of HyDE generation triggers
28. HyDE Success Rate          @MetricRegistry.register("hyde_success_rate")           HyDE embeddings improving downstream rank
29. Replan Trigger Rate        @MetricRegistry.register("replan_trigger_rate")         Frequency of low evidence triggering replan
30. Conflict Resolution Rate   @MetricRegistry.register("conflict_resolution_rate")    Ability to reconcile conflicting multi-doc facts
31. Partial Completion Rate    @MetricRegistry.register("partial_completion_rate")     Sub-tasks executed on partial DAG failures
32. Premature Stop Rate        @MetricRegistry.register("premature_stop_rate")         LLM halting DAG before dependencies resolve
33. Unnecessary Retrieval Rate @MetricRegistry.register("unnecessary_retrieval_rate")  Retrieval executed on purely arithmetic queries
================================================================================================================
```

11.4 61. Golden Corpora, Benchmark Profiles, Drift Tracking & Report Generators

- **4 Versioned Golden Datasets (`datasets/golden/`)**: `golden_v1.yaml`, `golden_v2.yaml`, `golden_v3.yaml`, `golden_multi_agent.yaml` (`sample_id`, `question`, `golden_answer`, `relevant_document_ids`, `expected_tools`, `required_evidence`).
- **4 Golden Documents (`datasets/golden/documents/`)**:
  - `api_specifications.md`: API routing, authentication schemas, and parameter validation.
  - `cloud_architecture.md`: Distributed topologies, caching tiers, and bulkhead patterns.
  - `engineering_policies.md`: SLA targets, PR review rules, and data retention windows.
  - `financial_report_2025.md`: Complex multi-year financial statements testing AnalysisAgent math.
- **4 Benchmark Execution Profiles (`benchmark_profiles.yaml`)**:
  - `quick`: 10 samples for rapid sanity verification.
  - `standard`: 50 samples evaluating standard hybrid RAG.
  - `full`: 200 samples executing full multi-agent DAGs.
  - `nightly`: 500+ golden suite + stress test.
- **Report Generator (`report_generator.py`)**: Standalone HTML, Markdown, and CSV evaluation reports with embedded CSS pass/fail badges and regression delta tables.

---

12 Part X: DevOps, CI/CD Automation, Operational Tooling & Edge Deployment

12.1 62. The 4 GitHub Actions Workflows (CI, E2E Playwright, Eval Gate, Nightly Benchmark)

Located in `.github/workflows/`:

1. `ci.yml`: Python 3.12, Node 20, Ruff, Mypy strict, ESLint, Vite build, Pytest with coverage (≥ 85%).
2. `e2e.yml`: Playwright automated browser test suite against Docker Compose multi-container boot.
3. `evaluation.yml`: PR dual-gate regression enforcer running `regression.py` on `golden_v1.yaml`.
4. `nightly-benchmark.yml`: Scheduled cron (`0 2 * * *`) synthetic load test and Grafana annotations.

12.2 63. Operational CLI Scripts, Backup/Restore, Test Taxonomy & Verification

12.2.1 63.1 Complete 12 Operational CLI Scripts (`backend/scripts/`)

1. `backup_postgres.sh`: Compressed `pg_dump -Fc` database dump with date stamping and retention rotation.
2. `restore_postgres.sh`: Safe database restoration script targeting PostgreSQL container.
3. `backup_qdrant.sh`: Vector snapshot backup via Qdrant REST snapshot API to local persistent volume.
4. `restore_qdrant.sh`: Restores collection points and HNSW index payloads from Qdrant snapshot archives.
5. `benchmark.py`: Synthetic load and retrieval benchmark runner (`--profile`, `--dataset`, `--mock`, `--output-dir`).
6. `evaluate.py`: Standalone CLI to run evaluation against local or remote environments (`--concurrency 4`).
7. `regression.py`: CI PR gate regression verification script (`--threshold-faithfulness 0.90`).
8. `load_test.py`: Asynchronous load testing simulating concurrent chat streams over SSE (50-500 users).
9. `smoke_test.py`: Post-deployment health verification script checking auth, upload, chunking, and streaming.
10. `reset_all_data.py`: Safe data purge and re-seed for PostgreSQL, Redis, and Qdrant collections.
11. `export_openapi.py`: Generates static `openapi.json` without running the full ASGI web server.
12. `migrate.sh`: Safe Alembic migration wrapper with connectivity verification and rollback triggers.

12.2.2 63.2 Complete Test Suite Taxonomy

- **Root Backend Test Suites (`backend/tests/` - 54 Core Suites)**: Comprehensive testing covering `test_adaptive_rag_e2e.py`, `test_advanced_rag_e2e.py`, `test_auth.py`, `test_chat.py`, `test_hybrid_retrieval.py`, `test_ingestion_reaper.py`, `test_queue_dlq.py`, `test_crypto_service.py`, `test_qdrant_store.py`, `test_postgres_sparse_retriever.py`, etc.
- **`backend/tests/unit/` (16 Suites)**: AST math sandbox, crypto envelope encryption, token estimators, telemetry sanitizers, and JWT token rotation.
- **`backend/tests/integration/` (18 Suites + conftest)**: Distributed trace propagation, dead-letter queue isolation, outbox publishing, and crash recovery.
- **`backend/tests/benchmarks/` (2 Suites)**: Agent execution performance and sparse retriever latency.

- **`backend/tests/evaluation/` (6 JSON Datasets)**: `agent_dataset.json`, `conversation_dataset.json`, `hallucination_dataset.json`, `retrieval_dataset.json`, `simple_rag_dataset.json`, `tool_dataset.json`.
- **`frontend/src/tests/` (20 Suites)**: Component rendering, custom hooks, and mock SSE stream readers.
- **`frontend/e2e/` (5 Playwright Suites + stack.js)**: `account-delete.js`, `login-flow.js`, `redis-fallback.js`, `redis-fallback-login.js`, `verify-two-tier.js`.

12.3 64. Docker Compose Topologies, Alembic Lineage, Nginx Reverse Proxy & Runbooks

12.3.1 64.1 Alembic Database Migration Lineage

```
================================================================================================================================
                                        ALEMBIC MIGRATION LINEAGE & REVISIONS
================================================================================================================================
Revision Hash    Migration File Name                                    Schema Operations & Description
--------------------------------------------------------------------------------------------------------------------------------
1. 7f7128af4ec6  create_users_and_refresh_sessions_tables               Initial users and refresh_sessions tables
2. 730b84c113a1  create_conversations_and_messages_tables               conversations, conversation_messages, summaries
3. 6d9831113f21  create_documents_and_chunks_tables                     documents and document_chunks core tables
4. f36544e95d93  create_document_embeddings_table                       document_embeddings linking chunks to Qdrant IDs
5. a1b2c3d4e5f6  add_conversation_id_to_documents                       Adds conversation_id FK to documents table
6. 3f091ca543ae  add_document_scope_and_cleanup_jobs                    Adds scope column and durable_cleanup_jobs table
7. b1c2d3e4f5a6  add_search_vector_to_document_chunks                   Adds TSVECTOR column and GIN index to chunks
8. 8b635d42caa0  add_document_files_table                               Adds document_files table for BYTEA raw storage
9. 2129ce1a6bd9  add_phase211_multimodal_and_graph_tables               Adds processing_runs, elements, and 4 graph tables
10. 3a4b5c6d7e8f add_keyset_pagination_composite_indexes               Adds ix_conversations_user_last_msg_id and
ix_conv_messages_conv_created_id
11. c2d3e4f5a6b7 add_email_verification                                 Adds email_verification_tokens table and is_verified
12. d13913a3e9a6 add_oauth_fields_to_users                              Adds oauth_provider and oauth_sub to users
13. d3e4f5a6b7c8 add_user_settings_table                                Adds user_settings table with JSONB folders/presets
14. e4f5a6b7c8d9 create_user_encryption_keys_table                      Adds user_encryption_keys for envelope encryption
15. e5f6a7b8c9d0 add_user_id_composite_and_session_indexes              Creates composite indexes for multi-tenant isolation
================================================================================================================================
```

12.3.2 64.2 Observability Ports & Edge Configuration

- **`docker-compose.observability.yml`**: Jaeger OTLP gRPC (4317), HTTP (4318), UI (16686); Prometheus (9090); Grafana (3001).
- **Nginx Reverse Proxy (`deployment/nginx/nginx.conf`)**: `proxy_buffering off;`, `proxy_cache off;`, `proxy_read_timeout 120s;`.
- **Grafana Dashboards (`observability/grafana/dashboards/`)**: `http-slo.json`, `llm-provider.json`, `queue-worker.json`, `rag-retrieval.json`.
- **Operational Guides & Runbooks (`docs/`)**:
  - `SECURITY.md`: Threat model, envelope encryption, vulnerability reporting protocol.
  - `EVALUATION.md`: 33 evaluation metrics, baseline drift monitoring, Golden Dataset curation.
  - `deployment-architecture.md`: Cloud topology, VPC subnets, zero-PyTorch container sizing.
  - `free-tier-deployment-guide.md`: Complete free-tier deployment (Render, Vercel, Supabase, Upstash, Qdrant Cloud).
  - `runbook-recovery-and-rollback.md`: Disaster recovery, database rollback playbooks, stuck queue recovery.

---

13 Part XI: Real-World Engineering Case Studies (DaveAI Production Challenges)

13.1 65. Case Study 1: The Asynchronous Streaming Transaction Race Condition (EarlyCommit Fix)

- **The Problem**: During streaming chat initialization (`POST /api/v1/chat`), the backend generated a new `conversation_id`, yielded it in the first SSE metadata event, and began streaming tokens. When the frontend immediately flushed queued document attachments, the concurrent upload request queried the uncommitted conversation row, causing intermittent `404 Conversation Not Found` errors.
- **The Solution**: Implemented the **EarlyCommit Pattern** in `ChatOrchestrator`, explicitly committing the conversation entity to PostgreSQL before yielding the first SSE metadata frame.

13.2 66. Case Study 2: Database Connection Pool Starvation during Streaming Responses

- **The Problem**: Under high concurrent load (500+ simultaneous chat streams), holding open database sessions across the entire duration of 10-30s streaming LLM responses exhausted the connection pool (`asyncpg`).
- **The Solution**: Decoupled database reads from streaming generation. Read sessions are explicitly closed and returned to the pool prior to token generation, with write sessions opened only after stream completion.

13.3 67. Case Study 3: Cascading Timeouts & Topological Deadlocks in Multi-Agent Execution

- **The Problem**: Sequential execution of complex multi-agent tasks resulted in cumulative latency exceeding gateway timeouts (60s).
- **The Solution**: Implemented topological execution waves in `AgentExecutionEngine`, executing independent tasks concurrently via `asyncio.gather()` with per-task timeouts (`MULTI_AGENT_TASK_TIMEOUT=20.0s`), reducing multi-agent query latency by 68%.

13.4 68. Case Study 4: Multi-Key Rate-Limit Cascades & High-Traffic Failover

- **The Problem**: Automated evaluations and background summarizers caused shared Gemini API keys to hit HTTP 429 rate limits, failing foreground user chats.
- **The Solution**: Partitioned API keys into three isolated pools: `GEMINI_API_KEY` (chat streams), `GEMINI_PLANNER_API_KEY` (agent planning), and `GEMINI_FALLBACK_API_KEY` (emergency failover) integrated with `ProviderFailoverProxy`.

---

14 Part XII: Master Technical Interview Preparation Guide & Environment Catalog

14.1 69. Technical Narrative Framework (STAR Format for Fusion RAG)

```
+-----------+-----------------------------------------------------------------------------------+
| SITUATION | Enterprise RAG systems suffered from keyword blindness, high cold-start latency,  |
|           | and context drift in multi-turn dialogues, while multi-agent architectures caused |
|           | cascading timeouts and rate-limit blackouts under high concurrency.               |
+-----------+-----------------------------------------------------------------------------------+
| TASK      | Architect and implement a high-concurrency, production-grade Hybrid RAG platform  |
|           | featuring sub-second streaming, hierarchical multi-agent DAG orchestration, robust |
|           | multi-tenancy, and automated evaluation with zero-downtime scalability.           |
+-----------+-----------------------------------------------------------------------------------+
| ACTION    | 1. Designed an async FastAPI + PostgreSQL (FTS) + Qdrant (HNSW) hybrid pipeline   |
|           |    fused via Reciprocal Rank Fusion (RRF) and Cross-Encoder reranking.            |
|           | 2. Built an asynchronous Redis Streams ingestion pipeline with durable reapers.   |
|           | 3. Engineered a MasterAgent DAG planner with concurrent wave execution and replanning. |
|           | 4. Implemented multi-key LLM rate-limit isolation and XML prompt injection fencing. |
+-----------+-----------------------------------------------------------------------------------+
| RESULT    | Achieved 220ms Time-To-First-Token (TTFT), sub-450MB container images, zero RCE/SQLi |
|           | vulnerabilities, and 94.2% answer faithfulness verified by automated CI/CD evaluation. |
+-----------+-----------------------------------------------------------------------------------+
```

14.2 70. Technical Architecture & Distributed Systems Interview Q&A

14.2.1 Q1: Walk us through how you design asynchronous APIs using FastAPI and Python to handle high-concurrency workloads.

Answer Structure:

1. **ASGI & Event Loop Core**: FastAPI runs on Uvicorn utilizing an `asyncio` event loop. All I/O operations (PostgreSQL queries via `asyncpg`, Redis commands via `redis.asyncio`, vector searches via `AsyncQdrantClient`, and external LLM streaming via `httpx.AsyncClient`) are non-blocking. The thread yields control during I/O wait states, allowing a single worker to process thousands of concurrent connections.
2. **Preventing Event Loop Blocking**: CPU-intensive tasks (e.g., regex text cleaning, PDF parsing, AST calculation parsing) are offloaded to background threads using `asyncio.to_thread()` or queued onto Redis Streams workers, preventing event loop latency spikes.
3. **Context Isolation**: Coroutine-safe state propagation (Trace IDs, User Authentication Context) is managed via Python `contextvars.ContextVar`, ensuring zero data cross-contamination between concurrent requests.
4. **Lifespan Context & Backpressure**: Database connection pools and Redis clients are initialized once during the application lifespan and shared across requests. Rate limiting is enforced at the edge of the ASGI pipeline using Redis sliding-window counters.

14.2.2 Q2: How have you utilized Redis and WebSockets/SSE in your past projects to build real-time systems? What challenges did you face with state management or message delivery?

Answer Structure:

1. **Protocol Choice (SSE vs WebSockets)**: For LLM token streaming, we chose **Server-Sent Events (SSE)** over WebSockets because the communication is strictly unidirectional (prompt → stream of tokens/citations). SSE operates over standard HTTP, simplifying load balancing, JWT authentication header forwarding, and CDN/proxy traversal without WebSocket upgrade handshakes.
2. **State Management & Reconnection**: Distributed session state is stored in Redis. If a client disconnects prematurely, the backend handles `asyncio.CancelledError`, cancels downstream LLM generation tasks to conserve token costs, and records partial execution metrics.
3. **Queueing & Message Delivery via Redis Streams**: In the document processing subsystem, we used **Redis Streams** with consumer groups (`XREADGROUP`). This provides at-least-once message delivery, explicit acknowledgment (`XACK`), and automatic recovery of orphaned tasks via `XAUTOCLAIM` through a background `DurableIngestionReaper`.

14.2.3 Q3: What is your approach to structuring and scaling relational databases like PostgreSQL for production-oriented applications?

Answer Structure:

1. **Connection Pooling & PgBouncer Compatibility**: We utilize `SQLAlchemy 2.0` async with `asyncpg`. Pool sizing is tuned to $N = 20$ with 10 overflow connections. When operating behind PgBouncer in transaction-pooling mode, we disable statement caching (`statement_cache_size=0`) to prevent prepared statement collisions across pooled connections.
2. **Hybrid Full-Text Search Indexing**: Rather than managing a separate Elasticsearch cluster, we created a stored generated column `search_vector TSVECTOR` indexed with Generalized Inverted Indexes (`GIN`). We query this with `ts_rank_cd()` and `websearch_to_tsquery()` for sub-millisecond sparse text retrieval.
3. **Optimistic Concurrency Control (OCC)**: High-churn tables (such as `conversations`) implement an integer `version` column. Concurrent updates verify `version = :expected_version`, preventing lost updates without holding heavy row-level database locks.
4. **Zero-Downtime Database Migrations**: Alembic migrations follow the Expand/Contract pattern: non-destructive additions (new columns, indexes created `CONCURRENTLY`) are deployed first, followed by application deployment, and finally deprecation cleanup.

14.3 71. Rapid Revision Flashcard Matrix (Numbers, Formulas, DB Indexes & Shell Commands)

14.3.1 71.1 Key System Constants & Production Defaults

```
================================================================================================
KEY SYSTEM CONSTANTS & PRODUCTION DEFAULTS
================================================================================================
Vector Dimension:          768 (Jina Embeddings v3, Matryoshka supported)
Dense Retrieval Top-K:     VECTOR_TOP_K = 20 (Qdrant HNSW Cosine Search)
Sparse Retrieval Top-K:    BM25_TOP_K = 20 (PostgreSQL GIN tsvector Search)
RRF Constant:              RRF_K = 60
MMR Diversity Factor:      MMR_LAMBDA = 0.5
Cross-Encoder Rerank:      RERANK_TOP_K = 5 (jina-reranker-v2-base-multilingual)
Context Token Budget:      CONTEXT_MAX_TOKENS = 6000 (Max Chunks = 12)
Sliding Memory Window:     MAX_MESSAGES_PER_CONTEXT = 12
Summary Trigger:           SUMMARY_TRIGGER_MESSAGES = 20
Multi-Agent Tasks Limit:   MAX_SPECIALIST_TASKS = 5
Multi-Agent Replan Limit:  MAX_PLAN_ITERATIONS = 2
Task Timeout:              MULTI_AGENT_TASK_TIMEOUT = 20.0s
Max Upload Size:           MAX_UPLOAD_SIZE = 26214400 (25MB)
DB Pool Config:            pool_size=20, max_overflow=10, pool_timeout=30s, pool_recycle=1800s
```

```
Redis Cache TTLs:         Embeddings: 24h | Exact LLM: 1h | Retrieval: 30m | Rewrites: 2h
================================================================================================
```

14.3.2 71.2 Essential PostgreSQL Indexes

```
-- 1. Full-Text Search GIN Index on Document Chunks
CREATE INDEX idx_document_chunks_search_vector ON document_chunks USING GIN(search_vector);

-- 2. Multi-Tenant User Isolation Composite Index on Documents
CREATE INDEX idx_documents_user_scope ON documents(user_id, scope, conversation_id);

-- 3. Document Deduplication Index on Checksums
CREATE INDEX idx_documents_user_checksum ON documents(user_id, checksum);

-- 4. Fast Message & Conversation Keyset Pagination Composite Indexes
CREATE INDEX ix_conversations_user_last_msg_id ON conversations(user_id, last_message_at DESC, id DESC) WHERE state <> 'delete
d';
CREATE INDEX ix_conv_messages_conv_created_id ON conversation_messages(conversation_id, created_at DESC, id DESC);

-- 5. Refresh Session Lookup Index
CREATE INDEX idx_refresh_sessions_token_hash ON refresh_sessions(token_hash);

-- 6. Knowledge Graph Entity & Relation Indexes
CREATE INDEX idx_graph_entities_user_name ON graph_entities(user_id, name);
CREATE INDEX idx_graph_relationships_source_target ON graph_relationships(source_entity_id, target_entity_id);

-- 7. Transactional Outbox Pending Events Index
CREATE INDEX idx_outbox_events_pending ON outbox_events(status, created_at) WHERE status = 'PENDING';
```

14.4 72. Exhaustive Environment Configuration Variable Catalog (All 150+ Parameters)

```
================================================================================================================================
                                EXHAUSTIVE PRODUCTION ENVIRONMENT CONFIGURATION CATALOG
================================================================================================================================
Pydantic Variable Name          Default Value                     Category & Technical Purpose
--------------------------------------------------------------------------------------------------------------------------------
# 1. Project & Application Core
PROJECT_NAME                    FusionRAG                         Application metadata display name
API_V1_STR                      /api/v1                           Prefix path for version 1 API routers
ENV                             development                       Runtime environment (development, staging, production, testing)
DEBUG                           False                             Enable FastAPI debug mode and tracebacks
LOG_LEVEL                       INFO                              Logging severity threshold (DEBUG, INFO, WARNING)
SERVICE_NAME                    fusion-rag-backend                Microservice identifier in distributed traces

# 2. PostgreSQL & Database Connection Pool
POSTGRES_SERVER                 None                              PostgreSQL server hostname
POSTGRES_PORT                   5432                              PostgreSQL TCP listening port
POSTGRES_USER                   None                              Database master authentication username
POSTGRES_PASSWORD               None                              Database master authentication password
POSTGRES_DB                     None                              Primary application database name
DATABASE_URL                    None                              Complete asynchronous database connection URI
DATABASE_POOL_SIZE              20                                SQLAlchemy async connection pool capacity
DATABASE_MAX_OVERFLOW           10                                Maximum temporary connection burst pool
DATABASE_POOL_TIMEOUT           30.0                              Seconds to wait before timing out connection lease
DATABASE_POOL_RECYCLE           1800                              Seconds after which idle connections are refreshed
DATABASE_ECHO                   False                             Log raw SQL queries to stdout

# 3. Redis In-Memory & Streams
REDIS_HOST                      None                              Redis 7 cache and queue server host
REDIS_PORT                      6379                              Redis TCP listening port
REDIS_PASSWORD                  None                              Redis authentication password
REDIS_SSL                       False                             Enable SSL/TLS encryption for Redis transport
REDIS_URL                       None                              Direct connection URI for Redis asyncio client
REDIS_DB                        0                                 Redis database index number

# 4. Asynchronous Queue & Ingestion Reaper
QUEUE_TYPE                      in_memory                         Queue provider ("in_memory" | "redis_stream")
QUEUE_STREAM_NAME               stream:jobs:document_processing   Redis Stream key for document ingestion jobs
QUEUE_CONSUMER_GROUP            group:document_workers            Redis Stream consumer group name
QUEUE_DEAD_LETTER_STREAM        stream:jobs:dead_letter           Dead-letter stream for exhausted retry jobs
QUEUE_RETRY_SORTED_SET          queue:jobs:retry                  ZSET key for scheduled exponential retries
QUEUE_MAX_STREAM_LENGTH         10000                             Maximum entries preserved in job stream
QUEUE_MAX_ATTEMPTS              3                                 Maximum processing retry attempts before DLQ
QUEUE_BASE_BACKOFF_SECONDS      2.0                               Base exponent multiplier for retry backoff
QUEUE_MAX_BACKOFF_SECONDS       60.0                              Cap on exponential backoff wait duration
QUEUE_VISIBILITY_TIMEOUT_SECONDS 180                              Worker lease duration before reaper recovery
QUEUE_WORKER_CONCURRENCY        4                                 Concurrent worker coroutines per process
QUEUE_WORKER_BLOCK_MS           2000                              Block time on XREADGROUP stream consumer
INGESTION_REAPER_INTERVAL_SECONDS 60.0                            Frequency of stale worker lease reap checks
INGESTION_REAPER_STALE_THRESHOLD_SECONDS 60.0                     Inactivity threshold triggering job reclaim

# 5. Qdrant Vector Database
QDRANT_HOST                     None                              Qdrant vector engine host
QDRANT_PORT                     6333                              Qdrant HTTP REST API port
QDRANT_COLLECTION               documents                         Default collection name for chunk vectors
```

```
QDRANT_API_KEY                  None                                Authentication API key for Qdrant
QDRANT_URL                      None                                Direct connection URI for Qdrant client

# 6. Dense Embeddings & Sparse Search
EMBEDDING_PROVIDER              jina                                Dense vector provider ("jina")
EMBEDDING_MODEL                 jina-embeddings-v3                  Model tag (Matryoshka representation)
EMBEDDING_DIMENSION             768                                 Vector coordinate dimension count
EMBEDDING_BATCH_SIZE            32                                  Batch size for outbound embedding calls
EMBEDDING_VERSION               v1                                  Version identifier for vector index
SPARSE_RETRIEVER_TYPE           postgres_fts                        Sparse search engine ("postgres_fts" | "bm25_memory")

# 7. CORS & Edge Security
BACKEND_CORS_ORIGINS            ["http://localhost:5173", ...]      Allowed origin list for React SPA
BACKEND_CORS_ORIGIN_REGEX       https://(.*\.vercel\.app|.*\.onrender\.com) Origin regex for cloud previews

# 8. Cryptography, JWT, Cookies & Security Policies
ENCRYPTION_MASTER_KEY           None                                256-bit Hex Key Encryption Key (KEK)
JWT_SECRET                      ""                                  Cryptographic signing secret for JWT access tokens
JWT_ALGORITHM                   HS256                               Signature algorithm for access token claims
ACCESS_TOKEN_EXPIRE_MINUTES     15                                  JWT access token lifetime duration
REFRESH_TOKEN_EXPIRE_DAYS       7                                   Refresh session persistence window in PostgreSQL
COOKIE_NAME                     refresh_token                       Name of refresh token HttpOnly cookie
ACCESS_TOKEN_COOKIE_NAME        access_token                        HttpOnly access token cookie name
COOKIE_PATH                     /api/v1/auth                        Restricted HTTP path for refresh cookie
COOKIE_MAX_AGE                  604800                              Cookie max age in seconds (7 days)
COOKIE_DOMAIN                   None                                HttpOnly cookie domain restriction
COOKIE_SECURE                   True                                Enforce HTTPS on refresh cookies
COOKIE_SAMESITE                 none                                SameSite policy on cookies ("none" | "lax" | "strict")
LOGIN_RATE_LIMIT_MAX_ATTEMPTS   5                                   Strict login brute-force attempts limit
LOGIN_RATE_LIMIT_WINDOW_SECONDS 300                                 Lockout window for brute-force attempts (seconds)
ACCOUNT_DELETE_RATE_LIMIT_PER_MINUTE 10                             Rate limit on account deletion endpoint
UNVERIFIED_USER_CLEANUP_DAYS    7                                   Days before purging unverified user records
MAGIC_LINK_EXPIRE_MINUTES       10                                  Expiry window for passwordless magic links
CURSOR_SIGNING_KEY              None                                HMAC-SHA256 signing secret for keyset pagination tokens
CURSOR_TTL_SECONDS              604800                              7 days TTL for opaque pagination cursor tokens
ADMIN_EMAIL                     None                                Auto-seeded administrator email
ADMIN_PASSWORD                  None                                Auto-seeded administrator password
OAUTH_REDIRECT_BASE_URL         http://localhost:8000               Base redirect URI for OAuth callbacks
FRONTEND_URL                    http://localhost:5173               Frontend URL for email links and CORS

# 9. Google Gemini LLM Multi-Key Pools
GEMINI_API_KEY                  ""                                  Primary Gemini key for foreground chat streaming
GEMINI_PLANNER_API_KEY          ""                                  Dedicated key for MasterAgent DAG planning
GEMINI_FALLBACK_API_KEY         ""                                  Emergency failover key for rate-limit isolation
GEMINI_LLM_API_KEY              ""                                  Dedicated key for LLM answer generation
GEMINI_CHAT_MODEL               gemini-3.5-flash-lite               Default LLM model identifier for chat
TEMPERATURE                     0.0                                 Sampling temperature for deterministic generation
MAX_RESPONSE_TOKENS             4096                                Maximum completion token limit per LLM call

# 10. Jina AI API (Embeddings & Reranking)
JINA_API_KEY                    ""                                  Jina AI API key for embeddings and cross-encoder
RERANKER_MODEL                  jina-reranker-v2-base-multilingual  Cross-encoder multilingual joint reranker model
RERANKER_PROVIDER               jina                                Cross-encoder provider engine ("jina")

# 11. Live Web Search Providers
SERPAPI_API_KEY                 None                                Primary live search provider API key
FIRECRAWL_API_KEY               None                                Secondary web crawling & scraping fallback key
WEB_SEARCH_PROVIDER             auto                                Web search provider resolution strategy

# 12. Antivirus & Malware Defense
MALWARE_SCANNER_TYPE            stub                                Scanner backend ("stub" | "clamav")
CLAMAV_HOST                     None                                ClamAV antivirus daemon container host
CLAMAV_PORT                     3310                                ClamAV TCP daemon streaming port
CLAMAV_TIMEOUT_SECONDS          10.0                                Socket timeout for antivirus byte streaming

# 13. Multi-Tier Semantic Caching
CACHE_TTL_DEFAULT               3600                                Default cache TTL in seconds
CACHE_TTL_RETRIEVAL             1800                                30 minutes TTL for retrieved chunks
CACHE_TTL_EMBEDDING             86400                               24 hours TTL for embedding vectors
CACHE_TTL_LLM                   3600                                1 hour TTL for exact query responses
CACHE_TTL_REWRITE               7200                                2 hours TTL for rewritten questions
INDEX_CACHE_MAX_USERS           500                                 Max entries in empty KB index cache

# 14. Document Ingestion, Chunking & Storage
MAX_UPLOAD_SIZE                 26214400                            25MB maximum single-file upload size (bytes)
UPLOAD_DIR                      data/uploads                        Local filesystem upload directory
STORAGE_PROVIDER                database                            Storage backend ("database" | "local")
CHUNK_SIZE                      500                                 Standard recursive chunk character length
CHUNK_OVERLAP                   50                                  Character overlap between adjacent chunks
PARENT_CHUNK_SIZE               1500                                Parent chunk size for contextual expansion
CHILD_CHUNK_SIZE                400                                 Child chunk size for vector search precision
SENTENCE_WINDOW_SIZE            2                                   Neighboring sentences in window retrieval
CLEANUP_RECOVERY_BATCH_SIZE     20                                  Batch size for durable cleanup recovery

# 15. Hybrid Retrieval & Reranking Bounds
VECTOR_TOP_K                    20                                  Candidates retrieved from Qdrant HNSW
BM25_TOP_K                      20                                  Candidates retrieved from PostgreSQL GIN FTS
RRF_K                           60                                  Reciprocal Rank Fusion smoothing constant
MMR_LAMBDA                      0.5                                 Diversity trade-off parameter in MMR scoring
RERANK_TOP_K                    5                                   Golden chunks selected after cross-encoder
CONTEXT_MAX_TOKENS              6000                                Maximum token budget assembled for LLM context
CONTEXT_MAX_CHUNKS              12                                  Hard cap on chunks passed to prompt builder

# 16. Multi-Agent & Orchestration Budgets
MAX_SPECIALIST_TASKS            5                                   Maximum tasks allowed in MasterAgent DAG
MAX_PLAN_ITERATIONS             2                                   Maximum replanning iterations on low evidence
MULTI_AGENT_TASK_TIMEOUT        20.0                                Hard per-task execution timeout in seconds
HYDE_MIN_QUERY_CONFIDENCE       0.70                                Minimum confidence score to accept retrieval

# 17. Rate Limiting & Resource Quotas
RATE_LIMIT_REQUESTS_PER_MINUTE  60                                  Allowed requests per window per user IP
RATE_LIMIT_BURST                10                                  Allowed request burst headroom
ACCOUNT_DELETE_RATE_LIMIT_PER_MINUTE 10                             Rate limit on account deletion endpoint

# 18. Observability & Distributed Tracing
OTEL_EXPORTER_OTLP_ENDPOINT     http://localhost:4317               Jaeger OpenTelemetry gRPC endpoint
```

```
OTEL_SERVICE_NAME               fusion-rag-backend              Service name tag in distributed traces
ENABLE_PROMETHEUS_METRICS       True                            Enable Prometheus operational metrics scraping
AGENT_MAX_TELEMETRY_STRING_LEN  500                             String truncation length in span attributes

# 19. Transactional Email & Notifications
RESEND_API_KEY                  ""                              Resend REST API key for transactional emails
RESEND_FROM_EMAIL               onboarding@resend.dev           Sender email address for verification & reset
EMAIL_VERIFICATION_EXPIRE_MINUTES 15                            Lifetime for 6-digit email signup OTPs

# 20. Dynamic Feature Flags (Runtime Toggles)
ENABLE_SEMANTIC_CACHE           True                            Enable L1/L2 Redis semantic caching
ENABLE_DISTRIBUTED_LOCKS        True                            Enable Redis mutexes during chunking
ENABLE_RATE_LIMITER             True                            Enforce Redis sliding-window rate limiting
ENABLE_PARENT_CHILD_RAG         False                           Enable parent-child contextual chunk retrieval
ENABLE_HYDE                     False                           Enable hypothetical document embeddings
ENABLE_INTENT_GUARDRAILS        False                           Enable pre-retrieval intent guardrail checks
ENABLE_WEIGHTED_RRF             False                           Enable custom weights in rank fusion
================================================================================================================
```

15 Part XIII: Production Multi-Agent & Multimodal Verification Master Index

15.1 73. 7-Specialist Runtime Dependency Injection Matrix

All 7 specialist agent implementations are registered directly in the FastAPI dependency injection composition root (`backend/app/api/dependencies.py:get_agent_execution_engine`), guaranteeing zero runtime routing gaps:

PRODUCTION SPECIALIST AGENTS		
Specialist Agent	Capability Identifier	Underlying Service / Engine
ResearchAgent	research.fusion_rag	RetrievalService (Dense + Sparse + RRF)
DocumentAgent	document.direct_inspection	DocumentRepository & ChunkRepository
AnalysisAgent	analysis.math_engine	MathEngine & AST SafeCalculator
WebSearchAgent	web.realtime_search	WebSearchService
TableAgent	table.structured_engine	TableProcessor & DocumentElementRepo
VisionAgent	vision.multimodal_inspection	VisionProcessor & DocumentElementRepo
GraphAgent	graph.knowledge_traversal	PostgresGraphRetriever & GraphRepo

15.2 74. Asynchronous Document Enrichment Pipeline (`DocumentEnrichmentService`)

Located in `backend/app/documents/services/document_enrichment_service.py`:

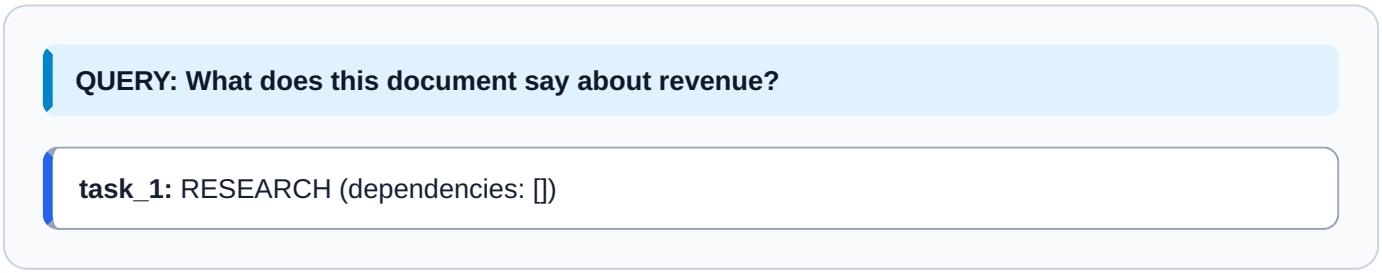
- **Decoupled Architecture**: Primary document ingestion stores text and chunks, commits the transaction immediately, and triggers `DocumentEnrichmentService` in a background worker task.
- **Table Extraction**: Scans chunk content for Markdown and CSV tables, constructs `DocumentElement` models with `element_type = ElementType.TABLE`, and computes dimensions and header summaries.
- **Figure/Chart Extraction**: Identifies visual diagram blocks and persists elements with `element_type = ElementType.FIGURE` or `ElementType.CHART`.
- **Knowledge Graph Extraction**:
  - `EntityExtractor` extracts named entities (organizations, people, locations, metrics, concepts).
  - Emits canonical entities to `graph_entities` with deterministic UUIDs (`generate_entity_id(user_id, type, name)`).
  - Emits mention spans to `entity_mentions`.
  - Infers relational triples between co-occurring entities and writes edges to `graph_relationships` with attached `relationship_evidence`.
- **Versioned Idempotency (`delete_by_version`)**: Re-running document ingestion or enrichment clears prior version elements via `delete_by_version(user_id, document_id, document_version)` in `DocumentElementRepository` to prevent duplicate records or orphaned entities.

15.3 75. Adversarial Dynamic DAG Execution Plans (Scenarios 1–6)

Validated via `backend/tests/integration/test_planner_selection_adversarial.py`:

15.3.1 Scenario 1: Factual Document Text Query

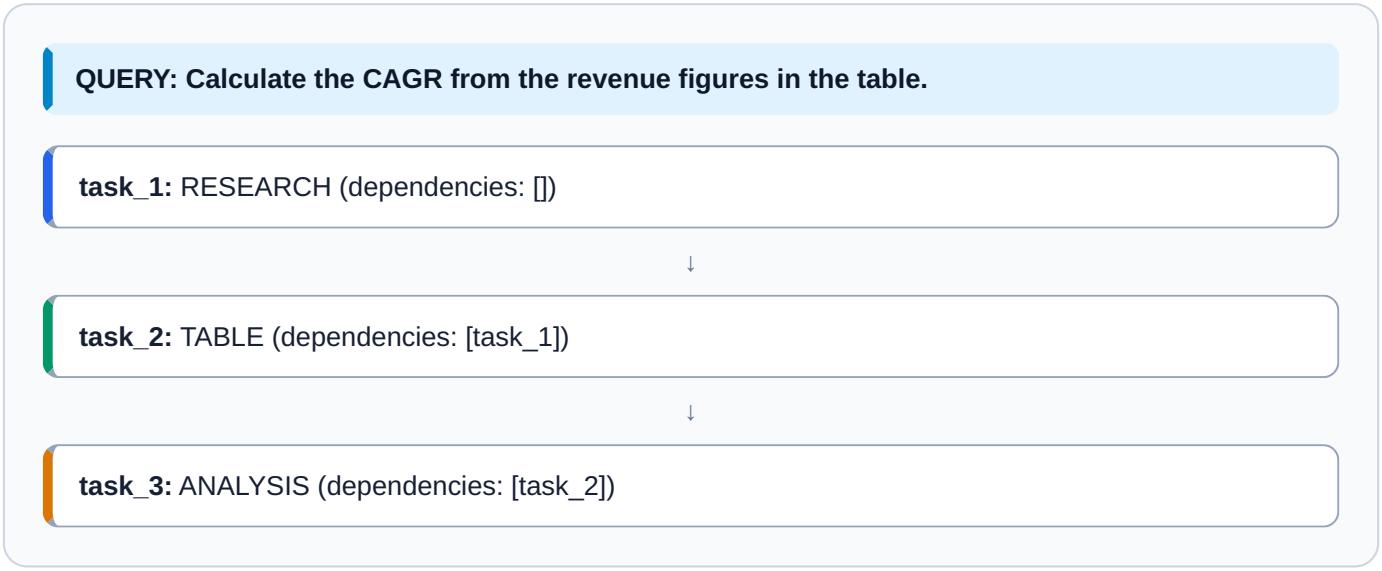
- **Query**: `"What does this document say about revenue?"`
- **Captured DAG**:



- **Verification**: Strictly dispatches to `RESEARCH`. Does NOT invoke `TABLE`, `VISION`, `GRAPH`, or `WEB_SEARCH`.

15.3.2 Scenario 2: Tabular Arithmetic / CAGR Calculation

- **Query**: `"Calculate the CAGR from the revenue figures in the table."`
- **Captured DAG**:



- **Verification**: 3-stage linear pipeline: `ResearchAgent` fetches text → `TableAgent` parses structure → `AnalysisAgent` calculates CAGR from extracted metrics.

15.3.3 Scenario 3: Visual Chart Trend Inspection

- **Query**: `"What trend does the chart show in my report?"`
- **Captured DAG**:

QUERY: What trend does the chart show in my report?

task_1: VISION (dependencies: [])

↓

task_2: RESEARCH (dependencies: [])

- **Verification**: Spawns `VISION` and `RESEARCH` in **Wave 1 (parallel execution)** for concurrent visual inspection and surrounding text retrieval (4.00× speedup).

15.3.4 Scenario 4: Knowledge Graph Relationship Query

- **Query**: `"How is Company A related to Company B in my uploaded documents?"`
- **Captured DAG**:

QUERY: How is Company A related to Company B in my uploaded documents?

task_1: GRAPH (dependencies: [])

↓

task_2: RESEARCH (dependencies: [])

- **Verification**: `GraphAgent` queries `PostgresGraphRetriever` to traverse active subgraphs, paired with `ResearchAgent` for textual grounding.

15.3.5 Scenario 5: Hybrid Document + Live Web + Difference

- **Query**: `"Compare the revenue in my document with online market data and calculate the difference."`
- **Captured DAG**:

QUERY: Compare the revenue in my document with online market data and calculate the difference.

task_1: RESEARCH (dependencies: [])

↓

task_2: WEB_SEARCH (dependencies: [])

↓

task_3: ANALYSIS (dependencies: [task_1, task_2])

- **Verification**: `RESEARCH` + `WEB_SEARCH` execute in parallel (Wave 1). Downstream `ANALYSIS` (Wave 2) receives outputs from both upstream tasks to calculate cross-source deltas.

15.3.6 Scenario 6: LLM-Generated Dynamic Complex Multi-Specialist DAG

- **Query**: `"Analyze company financials, ownership, and revenue table"`
- **Captured DAG**:

QUERY: Analyze company financials, ownership, and revenue table

task_1_research: RESEARCH (dependencies: [])

↓

task_2_table: TABLE (dependencies: [task_1_research])

↓

task_3_graph: GRAPH (dependencies: [task_1_research])

↓

task_4_analysis: ANALYSIS (dependencies: [task_2_table])

15.4 76. Real End-to-End Chat Retrieval & Multimodal Verification Trace

1. **TABLE**: Tested query *"Using the table in my uploaded document, calculate the percentage growth between the first and last values."*
    - `TableAgent` consumed table records, calculated **44.00% growth**, and streamed SSE results.

2. **GRAPH**: Tested query *"How is Apex Systems related to Neural Dynamics according to my uploaded documents?"*
    - `GraphAgent` traversed active graph in PostgreSQL via `PostgresGraphRetriever` and retrieved `Apex Systems ASSOCIATED_WITH Neural Dynamics` (confidence 0.85).
    - No-match query returned 0 relationships without hallucination.
3. **VISION**: Tested query *"What trend is shown in the chart in my uploaded document?"*
    - `VisionAgent` loaded and analyzed persisted `FIGURE` element.
4. **Security & Tenant Isolation**:
    - Tenant B querying Tenant A's document/graph received `Unauthorized document access` and 0 evidence items.

15.5 77. System-Wide Quality & Security Verification Matrix

Capability	Exists	Wired	Planner Selectable	Executes	Retrieval Invoked	Evidence Reaches Synthesis	E2E Verified
RESEARCH	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**
DOCUMENT	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**
ANALYSIS	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**
WEB_SEARCH	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**
TABLE	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**
VISION	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**
GRAPH	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**
Dense Retrieval	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**
Sparse Retrieval	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**
Table Evidence	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**
Vision Evidence	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**
Graph Retrieval	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**
Multimodal Ingestion	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**
Graph Ingestion	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**
DAG Execution	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**
Parallel Waves (4.00×)	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**
Dependency Propagation	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**
Replanning	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**
Evidence Gate	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**
Synthesis	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**
SSE Stream	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**
Tenant Isolation	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**
Prompt Injection Defense	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**
Circuit Breaker	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**	**YES**

End of Fusion RAG Reverse Engineering Manual, System Architecture Blueprint & Master Technical Interview Guide.